

TileBench: A Controlled Benchmark for Performance Evaluation and Bottleneck Diagnosis of Tile-Based Programming Models

Anonymous ACL submission

Abstract

Tile-based programming models, such as Triton and cuTile, aim to simplify high-performance kernel development, but their practical performance, tuning behavior, and usability remain difficult to compare systematically. We present TileBench, a controlled benchmark for evaluating Triton and cuTile on NVIDIA B200 GPUs under matched operator semantics and comparable implementation structures. TileBench contains 45 operators covering diverse AI-kernel patterns and memory/computation behaviors. Each task provides a PyTorch reference, verified Triton and cuTile implementations, standardized data-types (dtype) and input-size sweeps, default and autotuned configurations, roofline-based metrics, and profiling-guided diagnosis. Our evaluation shows that performance gaps are workload-dependent: cuTile excels on a small cluster of Tensor-Core/TMA-friendly kernels, while Triton is stronger on many irregular, streaming, and bandwidth-bound operators. We further evaluate LLM-generated cuTile and Triton kernels and find that Triton is consistently more token-efficient than cuTile under the same iterative refinement protocol.

1 Introduction

Modern AI workloads, such as Large Language Models (LLMs) (Brown et al., 2020), diffusion models (Ho et al., 2020), and multi-modal models (Radford et al., 2021), increasingly depend on custom kernels beyond standard library primitives, including attention variants (Dao, 2024), normalization (Zhang and Sennrich, 2019), Mixture-of-Experts (MOE) routing (Jiang et al., 2024), quantization and dequantization (Xiao et al., 2024), and fused point-wise operations (Lin et al., 2026), which involve diverse computation patterns. Although vendor libraries such as cuBLAS (NVIDIA, 2024a), cuDNN (Chetlur et al., 2014), template libraries such as CUTLASS (NVIDIA, 2024b), and

specialized kernels such as FlashAttention (Dao, 2024) provide highly optimized implementations for common primitives, many emerging operators still require custom kernel development involving custom fusion patterns, memory layouts, and advanced algorithms to fully exploit modern GPU hardware (Bentz, 2025; Lin et al., 2026).

Implementing high-performance GPU kernels remains difficult. CUDA (Nickolls et al., 2008) provides fine-grained control over the memory hierarchy, synchronization, and Tensor Cores, but this control comes with substantial engineering costs (Liao et al., 2025). High-level tile-based programming models such as cuTile (Bentz, 2025) and Triton (Tillet et al., 2019) aim to reduce this burden while offering efficient performance. Unlike the traditional Single Instruction Multiple Threads (SIMT) programming model (Lindholm et al., 2008), these domain-specific languages (DSLs) enable developers to program with a higher abstraction: they load and store chunks of data (Tiles) and apply certain computations on these tiles through tile-level primitives, facilitating the adoption of Tensor Cores (i.e., `mma`), Tensor Memory Accelerator (i.e., `cp.async.bulk.tensor`), and Tensor Memory instructions (i.e., `tcgen05`). The design philosophy of these DSLs enables developers to "focus on your algorithm" while handing complex code optimizations to the compiler and runtime.

Though such abstractions hide low-level complexity, it is still difficult for developers to diagnose performance bottlenecks and optimize the kernels for diverse AI operators. Recent cuTile evaluation studies (Yadav et al., 2026) compare cuTile with cuBLAS, Triton (Tillet et al., 2019), WMMA, and raw SIMT on GEMM and attention across Hopper and Blackwell GPUs, showing that cuTile can be highly effective on selected B200 workloads but is also strongly workload- and architecture-dependent. On the other hand, Triton can yield different performance with regard

to different versions of the compiler, autotuning space, ML algorithm, or implementation details when integrated into systems and libraries such as vLLM (Kwon et al., 2023), LightLLM (Hu et al., 2024), Liger-Kernel (Hsu et al., 2024), and Unsloth (Daniel Han and team, 2023). As a result, the community lacks a systematic understanding of performance bottlenecks, best practices for coding with these DSLs, and comparisons among different tile-centric DSLs.

To support fair evaluation and future tooling for tile-centric DSLs, we present **TileBench**, a controlled benchmark for comparing Triton and cuTile implementations of AI operators. TileBench contains 45 operators spanning attention and sequence primitives, GEMM variants, normalization, reductions, loss functions, elementwise and fused pointwise kernels, convolution and stencil computations, data movement, indexing, quantization, and sorting. Each task includes a PyTorch reference implementation for correctness checking and baseline measurement, together with matched Triton and cuTile implementations.

Our work makes the following contributions:

- We introduce TileBench, a controlled benchmark of 45 GPU operators on an NVIDIA B200 GPU, with manually written and verified Triton/cuTile implementations under matched semantics, comparable implementation structures, and standardized multi-dtype and input-size sweeps.
- We evaluate both default and autotuned configurations, quantifying how much performance is gained from systematic autotuning and how much performance headroom remains relative to hardware-aware limits.
- We build a profiling-guided analysis workflow to explain performance differences between Triton and cuTile using generated code, compiler behavior, and hardware-level metrics.
- We evaluate Triton and cuTile as target programming models for LLM-generated kernels, measuring not only correctness and runtime performance but also the token consumption required to achieve performant implementations.

Across the suite, both programming languages substantially outperform PyTorch, while Triton

achieves stronger overall performance, and cuTile is competitive on a small set of Tensor Core and TMA-friendly workloads. Autotuning improves both backends but leaves substantial roofline headroom, suggesting that compiler lowering and memory behavior often dominate tile-parameter choices. Evaluation on LLM-generated cuTile and Triton kernels demonstrates that Triton is consistently more token-efficient than cuTile under the same iterative refinement protocol.

2 Related Work

2.1 Tile-Based GPU Programming Models

Triton (Tillet et al., 2019) and cuTile (Bentz, 2025) are two representative tile-centric interfaces that are directly relevant to NVIDIA GPUs. Triton exposes a Python-embedded block program in which users specify tile sizes, pointer arithmetic, and scheduling hints, while the compiler lowers tile-level tensor operations to GPU code, and tensor descriptor. On NVIDIA GPUs with Tensor Memory Accelerator (TMA) support, Triton’s tensor descriptor API can lower descriptor-based loads and stores to TMA-backed memory (Triton Document). cuTile raises a similar abstraction inside the CUDA ecosystem: kernels are expressed through tile operations such as `ct.load`, `ct.mma`, and `ct.store`, and the compiler and runtime map these operations to threads, memory movement, Tensor Cores, and TMA.

The broader ecosystem also includes several production and research systems, such as TileLang (Wang et al., 2025), ThunderKittens (Spector et al., 2025), Tilus (Ding et al., 2026), TLX (Guan et al., 2026), and AWS NKI (AWS, 2026). However, these systems are not all directly comparable in our setting: NKI targets non-GPU accelerators, TLX extends Triton rather than serving as an independent high-level backend, and TileLang, ThunderKittens, and Tilus expose low-level parallelism, scheduling, layouts, and runtime controls. TileBench, therefore, focuses on Triton and cuTile: both can be evaluated on the same NVIDIA B200 platform, both expose tile-centric programming interfaces, and both support a meaningful apples-to-apples comparison under matched semantics, comparable tuning budgets, roofline analysis, and profiling-guided diagnosis.

2.2 Benchmarks for GPU Kernel Generation and Optimization

Recent benchmarks evaluate LLM-written or automatically optimized GPU kernels from different stages. KernelBench (Ouyang et al., 2025) and CUDABench (Zhu et al., 2026) focus on text-to-CUDA generation; BackendBench (Saroufim et al., 2025) and TritonBench (Li et al., 2025) study PyTorch backend or Triton operator generation; FlashInfer-Bench (Xing et al., 2026) emphasizes serving-oriented inference primitives; ComputeEval (NVIDIA, 2025) covers broader CUDA programming ability; and SOL-ExecBench (Lin et al., 2026) introduces hardware speedup-of-light evaluation. These benchmarks measure correctness, software-baseline speedup, GPU efficiency, or hardware headroom, but they do not isolate programming-model effects between tile-level DSLs. TileBench is complementary; its main track uses manually written and verified Triton/cuTile implementations under matched operator semantics and comparable autotuning ranges, while Section 3.3 separately measures how easy and token-efficient it is for LLMs to generate correct and performant kernels for each DSL.

3 TileBench

TileBench is a controlled benchmark for comparing Triton and cuTile under matched operator semantics and standardized evaluation. PyTorch serves as the semantic reference, correctness oracle, and software baseline, while Triton and cuTile are the primary programming models under comparison.

3.1 Overview and Pipeline

Figure 1 shows the TileBench workflow. The pipeline has three stages. First, the benchmark construction stage curates operators from TritonBench (Li et al., 2025) and LeetGPU (LeetGPU, 2025) and normalizes them into a unified TileBench task format. Second, the benchmark-execution stage evaluates each task through a shared harness: PyTorch produces reference outputs and baseline latency, while Triton and cuTile implementations are compiled, run through the configured default or autotuned path, verified, and timed on the same input cases. Third, the comparative analysis stage aggregates latency, speedup, throughput, effective bandwidth, and roofline utilization. For selected large-gap cases, we further use Nsight Compute (NVIDIA) together with

PTX/SASS (NVIDIA, 2026) inspection to analyze instruction mix, memory behavior, TMA/tcgen05 usage, and bank conflicts.

3.2 Benchmark Suite

Benchmark Sources and Coverage. TileBench contains 45 representative AI operator tasks derived from existing benchmark suites, including 26 from TRITONBENCH (Li et al., 2025) and 19 from LEETGPU (LeetGPU, 2025). We use these benchmarks as sources of operator semantics and task definitions, but all benchmark implementations are manually written and verified within the TileBench framework. For each operator, we provide controlled benchmark artifacts including `impl_torch.py`, `impl_triton.py`, `impl_cutile.py`, and `config.yaml`. This design ensures matched code shapes, algorithms, uniform data types (dtype) and input shapes, shared timing protocols, and consistent autotuning configurations across implementations. The main benchmark track evaluates manually implemented and verified Triton and cuTile kernels under the TileBench harness, while LLM-generated implementations are evaluated separately in Section 3.3.

The operators are grouped into five categories: *Point-wise*, *Reduction/Normalization*, *Matrix Multiplication/Attention*, *Stencil/Convolution*, and *Data Layout*. The category distribution is shown in Table 2. The full per-operator list is reported in Appendix A.1.

Dtype support and input-size sweeps are operator-specific. Each operator declares its supported dtypes and swept parameters in `config.yaml` (Appendix A.3), and TileBench evaluates 20 input configurations per supported dtype. This provides each task with standardized multi-dtype and multi-size coverage, designed to capture small-to-large scaling, B200 roofline behavior, and LLM-like regimes rather than inheriting the fixed cases from TritonBench or LeetGPU.

Standardization across Benchmarks. Standardization is necessary because the source benchmarks use different evaluation protocols: TritonBench provides Triton-focused operator-generation tasks and performance scripts, while LeetGPU follows a challenge-style format with per-problem functional tests and a single large performance case. TileBench reconstructs these operators into a unified benchmark with matched Triton/cuTile implementations, shared correctness checks, standard-

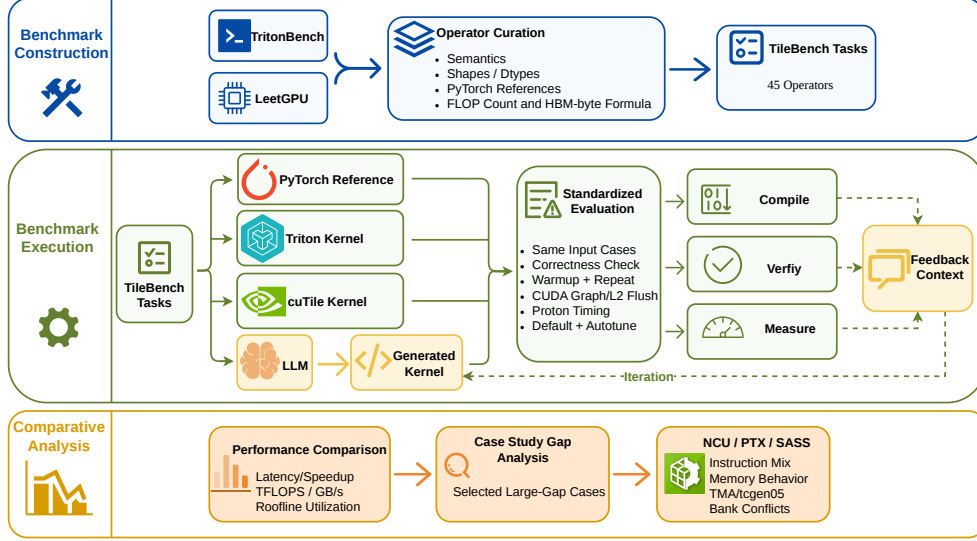


Figure 1: TileBench Overview.

ized multi-dtype and input-size sweeps, common timing controls, and comparable autotuning.

First, every operator has a PyTorch reference implementation that defines the semantic ground truth. Triton and cuTile outputs are compared against this reference using a shared correctness checker: floating-point outputs use `torch.testing.assert_close` with dtype- and operator-specific tolerances, while integer outputs use exact comparison (`atol = rtol = 0`).

Second, each operator has a `config.yaml` file (Appendix A.3) that declares the dtype list, case grid, verification tolerances, timing options, and analytical FLOP/HBM-byte formulas. The same formulas are evaluated for every (case, dtype) pair to derive throughput, effective bandwidth, arithmetic intensity, and roofline utilization consistently across backends.

Third, all implementations are measured by the same timing protocol and report the mean kernel latency. Measurement uses the Proton profiler (Zhou et al., 2026) with CUDA graph capture-and-replay, L2-cache flushing, 20 warmup runs, and 100 timed runs; NVIDIA Nsight Compute (NCU) is reserved for selected post-hoc case studies.

Fourth, in the main benchmark track, the Triton and cuTile implementations provide both a default execution path and an autotuned execution path. The default path uses a single fixed, manually selected configuration, while the autotuned path searches over a manually defined set of candidate configurations and records the winning one for analysis. Since Triton and cuTile expose dif-

ferent tuning knobs, TileBench uses comparable tuning ranges rather than a one-to-one parameter mapping. This design allows each backend to use its native tuning interface while keeping the autotuning budget and search granularity comparable across programming models.

3.3 Iterative LLM Code-Generation

TileBench further defines an iterative LLM code-generation track over the same operator set. For each operator, the LLM generates Triton and cuTile implementations from the natural-language operator description and TileBench task specification; it does not see or optimize the human-written implementations used in the main benchmark track. Unlike the main benchmark track, LLM-generated code is evaluated without backend autotuning, and each iteration must commit to a single configuration. Each backend is evaluated for 10 refinement iterations on one representative large case per supported dtype, with verification and performance feedback used to guide the next implementation and configuration.

At iteration 0, the prompt (Appendix B.1) is assembled from the TileBench framework guide (Appendix B.3), Triton and cuTile API references (Appendix B.4), the natural-language operator description (Appendix B.5), the operator `config.yaml` (Appendix A.3), the PyTorch reference implementation (Appendix B.6), and strict output-format instructions (Appendix B.7). The framework guide defines the implementation contract, multi-dtype handling, and hardware constraints. For iteration

$N > 0$, the prompt (Appendix B.2) is rebuilt with the previous trajectory, per-backend configuration, verification status, roofline utilization, speedup over PyTorch, worst-first performance summary, previous generated source files, and, when the latest attempt regresses or fails verification, the best verify-clean source files as a rebuild reference.

The evaluator rejects three classes of reward hacking before timing: delegating computation to PyTorch, cuBLAS, cuDNN, or the PyTorch reference implementation; caching outputs using tensor identity, data pointer, or version-dependent keys; and bypassing the intended search process by importing backend autotuners. These cases are checked using static scans, two-pass anti-cache verification, and rotating-input timing, and failures are returned to the LLM as structured feedback in the next iteration.

Triton and cuTile are tracked independently during refinement. The final result for each backend is selected as the best verify-clean iteration over the full trajectory.

3.4 Evaluation Metrics

Performance Metrics. For each operator o , backend b , dtype d , and input case c , the benchmark records mean kernel latency $T_{o,b,d,c}$. We report speedup over PyTorch as $T_{o,\text{torch},d,c}/T_{o,b,d,c}$, along with achieved throughput, effective bandwidth, arithmetic intensity, and roofline utilization. Let $F_{o,d,c}$ and $B_{o,d,c}$ be the analytical FLOP count and HBM-byte count declared in `config.yaml`. Achieved throughput is computed as

$$\text{TFLOPS}_{o,b,d,c} = \frac{F_{o,d,c}}{T_{o,b,d,c}} \cdot 10^{-12}.$$

Arithmetic intensity is $AI_{o,d,c} = F_{o,d,c}/B_{o,d,c}$. Roofline utilization normalizes achieved throughput by the dtype-specific roofline bound:

$$R_{o,b,d,c} = \frac{\text{TFLOPS}_{o,b,d,c}}{\min(P_d^{\text{peak}}, AI_{o,d,c} \cdot BW^{\text{peak}})}.$$

Here P_d^{peak} denotes the B200 peak throughput for dtype d , and BW^{peak} denotes the B200 peak HBM bandwidth; both constants are specified in `B200.json` (Appendix A.4).

Token-aware Metrics. For the iterative LLM code-generation track, let o denote an operator, $b \in \{\text{Triton}, \text{cuTile}\}$ a backend, $i \in \{0, \dots, 9\}$ a refinement iteration, and $\mathcal{D}_o$ the supported dtypes

evaluated in this track. For each (o, b, i) , it records the backend-level token usage $C_{o,b,i}^{\text{tok}}$, counting failed iterations because compilation failures and incorrect generations are part of the search process.

We use correctness-gated speedup to measure useful performance. Let $\text{correct}_{o,b,i}$ indicate that backend b at iteration i compiles and passes correctness checks for all evaluated dtypes. For dtype d , let $T_{o,\text{torch},d}$ be the PyTorch reference latency and $T_{o,b,i,d}$ be the generated implementation latency on the representative large case. The correctness-gated speedup of iteration i is

$$G_{o,b,i} = \begin{cases} \frac{1}{|\mathcal{D}_o|} \sum_{d \in \mathcal{D}_o} \frac{T_{o,\text{torch},d}}{T_{o,b,i,d}}, & \text{if } \text{correct}_{o,b,i} = 1, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, incorrect or non-compiling iterations contribute zero speedup while still incurring token costs. We report:

$$\text{BestSpeedup@10}_{o,b} = \max_{0 \leq i < 10} G_{o,b,i}.$$

The token cost is accumulated over the 10-iteration budget:

$$\text{TokenCost@10}_{o,b} = \sum_{i=0}^9 C_{o,b,i}^{\text{tok}}.$$

Token efficiency is the best correctness-gated speedup per million tokens:

$$\text{TokenEfficiency@10}_{o,b} = \frac{\text{BestSpeedup@10}_{o,b}}{\text{TokenCost@10}_{o,b}/10^6}.$$

When reporting aggregate results, we macro-average BestSpeedup@10 , TokenCost@10 , and $\text{TokenEfficiency@10}$ across operators for each backend.

4 Experimental Setup

We run two evaluations on TileBench: a main benchmark track comparing the human-written Triton and cuTile implementations of all 45 operators, and the iterative LLM code-generation track described in Section 3.3.

Hardware. All measurements are taken on a single NVIDIA B200 GPU with 180 GB of HBM3e memory; the measured peak bandwidth used for roofline analysis is 6539.4 GB/s.

Software and Sampling. We use PyTorch 2.10 with CUDA 13.0, Triton 3.6.0, and the cuda-tile 1.3.0 release. Per-operator FLOP count and HBM-byte counts are evaluated from the `flops_expr` and `bytes_expr` declared in each `config.yaml`, ensuring throughput, bandwidth, and roofline utilization are derived consistently across backends. For aggregated metrics, we use arithmetic mean over cases within each operator, and geometric mean over operators unless stated otherwise.

Models. For the iterative LLM track we evaluate two reasoning models: OpenAI GPT-5.5 (OpenAI, 2026) and Anthropic Claude Opus 4.7 (Anthropic, 2026), accessed through their respective APIs with default decoding settings.

5 Evaluations

- **RQ1: Autotuned Programming-Model Performance.** Under matched operator semantics and comparable implementation structures, how do the autotuned Triton and cuTile implementations compare with each other and with PyTorch baselines?
- **RQ2: Autotuning Sensitivity and Performance Headroom.** How much performance do the exposed configuration knobs recover beyond hand-selected defaults, and how much hardware headroom remains after tuning?
- **RQ3: Performance Gap Diagnosis.** Across TileBench, what differences in implementation structure and library dispatch explain the performance gaps between PyTorch and the DSL kernels, and what compiler lowering and hardware execution behaviors explain the remaining gaps between aligned Triton and cuTile implementations?
- **RQ4: LLM Usability and Token Efficiency.** Under a matched generation and feedback protocol with a maximum budget of 10 refinement iterations and no backend autotuning, how do Triton and cuTile compare in correctness-gated speedup, token cost, and token efficiency for LLM-generated kernels?

RQ1 reports the autotuned performance over the complete dtype and input-size sweeps. RQ2 directly compares the default and autotuned results over the same sweep. RQ3 profiles the NCU-catalogued maximum input of each supported

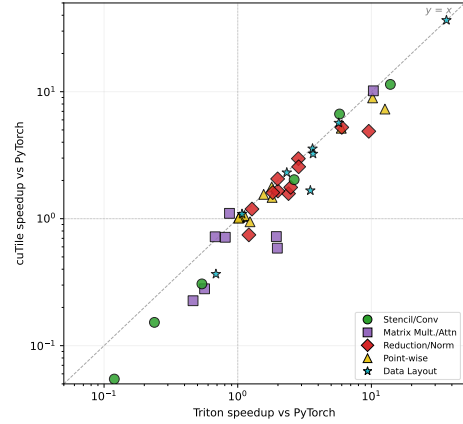


Figure 2: Triton and cuTile speedup over PyTorch across operators.

dtype using the autotuned winner selected for that exact case. RQ4 is evaluated separately: each model-backend pair is allowed up to ten refinement iterations and cannot invoke a backend autotuners; once a backend satisfies the stopping criterion, it is frozen and incurs no additional token cost. Unless otherwise stated, all aggregation above the per-case latency uses geometric means.

5.1 RQ1: Autotuned Programming-Model Performance

Answer: Under autotuned configurations, Triton is stronger in aggregate and more robust across diverse workloads, while cuTile remains competitive and achieves its clearest advantages on regular, reuse-heavy tiled workloads, including Tensor-Core / TMA kernels.

Figure 2 compares the operator-level autotuned speedups of Triton and cuTile over PyTorch. Each point summarizes one operator using the geometric mean over all of its supported dtype and input-size cases. We make three observations:

(1) **Both DSLs outperform PyTorch on most operators, but Triton is stronger overall.** Triton achieves a geometric-mean speedup of $2.02\times$ over PyTorch, compared with $1.58\times$ for cuTile. Triton is faster than PyTorch on 36/45 operators, while cuTile is faster on 33/45. Their median speedups are $1.95\times$ and $1.58\times$, respectively, showing that the aggregate difference is not caused only by a few outliers.

(2) **The direct Triton and cuTile comparison is workload-dependent.** Triton is faster on 37/45 operators, while cuTile is faster on 8/45. However, the backends remain within 5% on 14/45 operators and within 10% on 17/45,

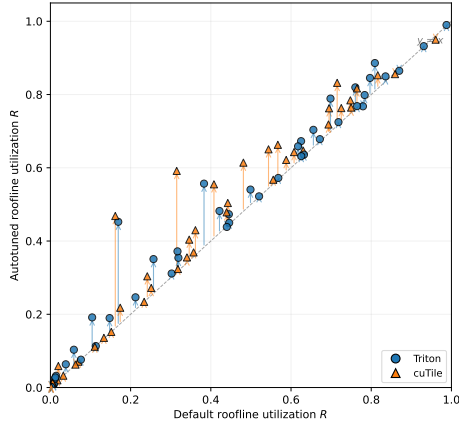


Figure 3: Default-to-autotuned roofline utilization across operators.

indicating that many head-to-head differences are small. The clearest cuTile advantages occur when regular tile movement, larger Tensor Core tiles, or two-dimensional data reuse can amortize tile-staging costs. Representative cases include `matmul_fp32_fp16_fp8`, `flash_decode`, and `jacobi_stencil_2d`. In contrast, Triton’s largest margins occur on runtime-computed indexing, dynamic scatter, fine-grained reductions, sparse traversal, and low-parallelism loops, as observed in `destindex`, `weight_dequant`, `moe_topk_gating`, `flash_decode`, and the convolution family.

(3) Speedup over PyTorch does not always isolate a compiler-backend effect. PyTorch serves as a practical software baseline, but its internal execution path varies across operators. For eager compositions such as `rope`, `dropout`, `kl_divergence`, `fused_activation`, the DSL advantage mainly comes from kernel fusion and avoiding intermediate tensors. For reductions and indexing operators, such as `mean_reduction`, `argmax`, `destindex`, the difference reflects a shape-specialized DSL kernel compared with general ATen implementation. Conversely, PyTorch can be faster when it dispatches to highly optimized vendor implementation, including cuDNN convolution, cuBLAS GEMM, CUB radix sort, and warp-specialized attention kernels.

Additional distributions and category-level results are reported in Appendix D.1.

5.2 RQ2: Autotuning Sensitivity and Performance Headroom

Answer: Autotuning provides meaningful additional gains beyond hand-selected defaults for

both backends, with a larger geometric-mean gain for cuTile, but substantial hardware headroom remains outside the exposed configuration spaces.

Figure 3 compares each operator’s geometric-mean default and autotuned roofline utilization. The geometric-mean improvement is $1.18\times$ for Triton and $1.22\times$ for cuTile, while the median improvements are $1.07\times$ and $1.06\times$, respectively. These gains are measured relative to manually selected defaults rather than random or deliberately weak configurations. Autotuning therefore recovers meaningful performance from tile shape, thread organization, pipeline depth, and occupancy choices that are difficult to select uniformly across operators and dtypes.

The autotuned results should be interpreted as the best observed configurations within the declared search spaces. Triton exposes block dimension, warp counts, pipeline stages, and operators-specific scheduling parameters, while cuTile exposes tile dimensions, occupancy hints, and operator-specific launch choices. The search does not alter the source-level algorithm or explicitly explore alternative implementation structures, primitive choices, or lowering strategies. RQ2 therefore measure *configuration headroom*, not a global upper bound on either programming model.

After tuning, 7/45 Triton implementations and 5/45 cuTile implementations reach at least 80% operator-level geometric-mean roofline utilization. Only 2/45 Triton implementations and 1/45 cuTile implementation reach at least 90%. Thus, autotuning closes part of the performance gap while also revealing substantial remaining *backend headroom*. These factors cannot generally be corrected by selecting another tile size from the same implementation.

5.3 RQ3: Performance-Gap Diagnosis

Answer: Performance gaps between PyTorch and DSL kernels are frequently caused by fusion, shape- and operator-specific kernel specialization, library dispatch, or algorithm organization. After the Triton and cuTile implementations are aligned, the remaining gaps arise from recurring differences in index lowering, reduction granularity, state placement, and pipeline scheduling. cuTile’s tensor core backend is not inherently weaker: it is highly effective on regular GEMMs, while its largest deficits occur when irregular indexing or fixed tile-lowering

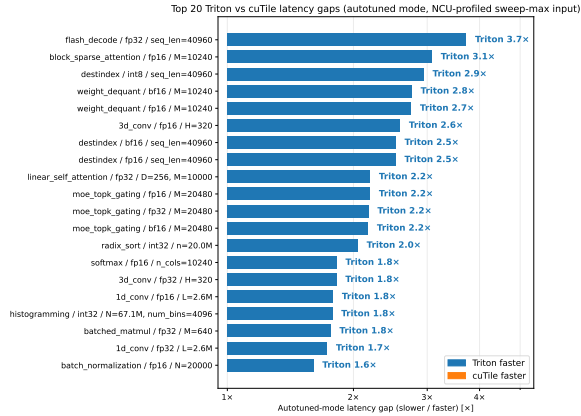


Figure 4: The 20 largest Triton-over-cuTile latency gaps under autotuned mode, evaluated at the NCU-profiled sweep-max input for each operator–dtype pair. Each bar represents one operator–dtype pair. The x-axis reports the ratio of the slower-backend latency to the faster-backend latency.

costs cannot be sufficiently amortized.

PyTorch versus the DSL kernels. PyTorch does not use one uniform execution path across TileBench. Its performance may reflect eager multi-kernel execution, a general ATen implementation, a compiled general-purpose operator, or a highly optimized vendor-library kernel. Consequently, the PyTorch comparison must first be interpreted through the implementation path. For example, the DSL speedups on dropout, rope, and kl_divergence primarily come from fusion and reduced intermediate traffic. By contrast, PyTorch advantages for the convolution family, radix_sort, flash_attention stem from specialized library algorithms and kernel organizations. Case such as block_sparse_attention lie between these extremes: compiled flex_attention follows the same parse block layout, while the DSL implementations expose additional opportunities for the fixed CSR traversal, tile sizes, and GQA layout, although the resulting backend performance still differs substantially.

Aligned Triton versus cuTile. Figure 4 shows that all 20 of the largest sweep-max gaps favor Triton. The largest are flash_decode/FP32 at 3.72 $\times$, block_sparse_attention/FP16 at 3.08 $\times$, and destindex/INT8 at 2.94 $\times$. The largest cuTile advantage, 1.55 $\times$ on FP32 matmul_fp32_fp16_fp8, falls below the twentieth Triton-faster gap and is analyzed separately below. Overall, the largest Triton advantages are concentrated in irregular in-

dexing, dynamic scatter, small reductions, sparse or runtime-controlled loops, and cases where fixed tile-management costs are large relative to useful work. The following two cases expose the contrast between irregular indexing and regular tiled computation.

Case A: index lowering in weight_dequant and destindex. To investigate compiler-level performance discrepancies, we profile two memory-bound kernels—*BF16 Weight Dequantization* and *Int8 Destination Indexing (Destindex)*—on B200. Despite their memory-bound nature, Triton outperforms Cutile by 2.98 $\times$ and 2.88 $\times$, respectively. Nsight Compute profiling reveals an artificial bottleneck inversion in Cutile: in BF16 Dequantization, Cutile saturates the SM (87.8%) and ALU (89.7%) pipelines while LSU utilization drops to 12.8%, whereas Triton maintains a balanced profile (31.6% SM, 33.8% ALU, and 25.8% LSU). A similar pattern occurs in Int8 Destindex, where Cutile sustains 79.4% ALU utilization versus 13.0% in Triton. While Triton warps predominantly encounter stall_long_scoreboard (hiding memory latency), Cutile warps are severely congested by stall_not_selected and stall_math_pipe_throttle, indicating that issue slots are choked by excessive integer arithmetic rather than memory stalls.

This compute saturation originates from severe instruction bloat during offset calculations rather than memory operations; while both compilers execute identical global load instructions, Cutile executes 6.2 $\times$ more total instructions in BF16 Dequantization and 17.3 $\times$ more in Int8 Destindex (86.99M vs. 5.04M). SASS disassembly traces this to multi-dimensional indexing (e.g., offsets // N and offsets % N): Triton lowers integer division/modulo via arith.divsi/remsi into constant reciprocal multiplications with magic constants (e.g., 0x66666667). Conversely, Cutile lowers index arithmetic into defensive “safe-division” routines spanning much more SASS instructions (IMAD, SHF, ISETP, LOP3, and SEL) to guard against signed inputs. Because tensor shapes and block offsets are strictly non-negative compile-time constants (tl.constexpr), Cutile’s lack of value-range propagation introduces entirely redundant arithmetic overhead.

Additionally, Cutile exhibits degraded memory coalescing in the Destindex kernel, achieving only 1.78 sectors per load request

(sectors_sum/requests_sum) compared to Triton’s 8.50. This disparity shows that Triton vectorizes contiguous thread accesses into wide 128-byte transactions, whereas Cutile fragments loads into disjoint, uncoalesced requests. In summary, the compounding effects of defensive arithmetic instruction bloat and memory transaction fragmentation fully account for Cutile’s performance degradation.

Case B: regular tiled computation in matmul_fp32_fp16_fp8. Matrix multiplication provides a control case because the irregular indexing chain disappears. Both implementations use grouped tiled GEMM, FP32 accumulation, Tensor-Core MMA, and TMA-backed memory movement. The performance difference therefore cannot be explained by cuTile using TMA while Triton does not.

NCU instead shows that cuTile keeps the Tensor pipeline active for approximately 79%–81% of cycles, compared with roughly 52%–69% for Triton. At $M = N = 4096$ and $K = 20480$, cuTile is $1.55\times$ faster in FP32, $1.12\times$ faster in FP16, and $1.22\times$ faster in FP8. The FP32 difference is largest because cuTile sustains a $256 \times 256 \times 64$ tile, while Triton’s TMA store staging and multi-stage buffers constrain its selected configuration to $128 \times 128 \times 32$. The smaller Triton tile nearly doubles the measured TMA load volume, 21.5 GB compared with 10.7 GB, while leaving scheduling and data-movement overhead amortized over less MMA work.

Cross-operator patterns. The two cases show that cuTile is not limited by weak Tensor-Core support. For regular GEMMs, it can feed the Tensor pipeline more effectively than Triton. Its larger deficits instead recur in five situations.

First, dynamic indexing, gather, and scatter can produce long integer-instruction and staging chains. This pattern appears in weight_dequant, destindex, radix sort, max pooling, and the convolution family.

Second, fine-grained reductions such as cross_entropy and moe_topk_gating expose cuTile’s minimum CTA granularity and shared-memory reduction trees, while Triton can use single-warp register and shuffle paths. Primitive availability also matters: operators such as top_k_selection and histogramming cannot be interpreted solely as compiler-code-generation comparisons because the two DSLs expose

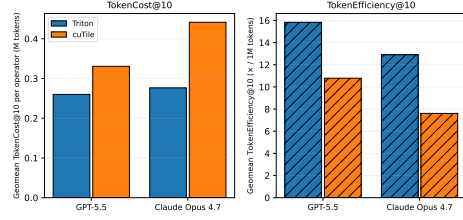


Figure 5: LLM token cost and efficiency by backend.

different specialized primitives.

Third, the two backends place tile state differently. Triton’s register-centric lowering is often more efficient for narrow dtypes, whereas cuTile’s shared-memory-centric lowering can support larger or more reusable tiles in matmul, flash_attention, and jacobi_stencil_2d.

Fourth, runtime-controlled loops in flash_decode, linear_self_attention, and, particularly at smaller shapes, streamk_matmul expose differences in software-pipeline depth and dependency handling.

Fifth, fixed per-tile bookkeeping, predicate handling, and conversion costs become more visible as the element size decreases. This explains dtype-dependent gaps in operators such as sigmoid, swiglu, and relu. In contrast, regular affine kernels that remain deeply bandwidth-bound, including matrix_copy, dropout, and fused_activation, provide near-parity control cases.

The convolution family provides a dimensional scaling study of the dynamic-indexing pattern and is reported in . Additional cross-operator results, including primitive availability, dtype-dependent overhead, backend-parity controls, and bank-conflict analysis, are reported in Appendix D.

5.4 RQ4: LLM Usability and Token Efficiency

Answer: Under the same maximum 10-iteration protocol, Triton is a more effective and token-efficient target for both evaluated LLMs. It produces verify-clean kernels for more operators, reaches higher correctness-gated speedup, and consumes fewer tokens than cuTile.

Figure 5 reports the geometric-mean TokenCost@10 and TokenEfficiency@10 for each model-backend combination. We additionally report the geometric-mean BestSpeedup@10 over operators with at least one verify-clean result to separate final performance from token expenditure.

For GPT-5.5, Triton produces a faster-than-PyTorch verify-clean kernel on 38/45 operators. cuTile does so on 36/43 operators with at least one verify-clean result, with two operators failing to produce a verify-clean cuTile implementation. The geometric-mean BestSpeedup@10 is $4.12\times$ for Triton and $3.46\times$ for cuTile. The corresponding token costs are 0.260M and 0.331M tokens, while token efficiency is $15.83\times$ and $10.79\times$ per million tokens, respectively.

For Claude Opus 4.7, Triton produces a faster-than-PyTorch verify-clean kernel on 38/45 operators. cuTile does so on 33/41 operators with at least one verify-clean result, with four operators failing to produce a verify-clean cuTile implementation. The geometric-mean BestSpeedup@10 is $3.57\times$ for Triton and $3.16\times$ for cuTile. The token cost increases from 0.277M tokens on Triton to 0.441M on cuTile, while token efficiency decreases from $12.90\times$ to $7.61\times$ per million tokens.

The backend effect is consistent across both models. Relative to cuTile, Triton improves token efficiency by approximately 47% for GPT-5.5 and 70% for Claude Opus 4.7. The difference comes from both sides of the metric: Triton more often reaches a stronger verify-clean implementation, and the refinement process consumes fewer tokens while doing so. The model effect is less uniform. GPT-5.5 achieves the strongest Triton result, while the differences between the two models are smaller than the corresponding Triton and cuTile gaps.

These results establish an empirical usability difference under the TileBench protocol, but they do not isolate its cause. Possible explanations include differences in API expressibility, available primitives, compiler constraints, documentation, examples, and model exposure to each DSL. Because the experiment does not separately control these factors, we treat reduced model familiarity with cuTile as a possible explanation rather than a demonstrated causal mechanism.

6 Conclusions

We introduce TileBench, a controlled and reproducible benchmark for comparing tile-based GPU programming models and studying tile-centric DSLs. Our results show that both Triton and cuTile substantially outperform PyTorch, but neither abstraction is uniformly superior: Triton is more robust on irregular, streaming, and bandwidth-bound operators, while cuTile excels when workloads

match static tile abstractions, Tensor Core computation, and reusable tile movement. Autotuning yields measurable gains, yet most kernels remain far from roofline limits, suggesting that backend lowering, memory staging, instruction selection, and shared-memory behavior matter more than tile-size search alone. The LLM-generated track further shows that programming-model design affects both final performance and the token cost of reaching correct, efficient kernels.

Limitations

TileBench currently targets a single NVIDIA B200 GPU and two tile-based DSLs. Its roofline metrics use analytical FLOP/HBM-byte formulas, so they provide a standardized hardware-aware summary rather than a replacement for full NCU-counter analysis.

Acknowledgments

References

- Anthropic. 2026. [What’s new in Claude Opus 4.7](#).
- AWS. 2026. [Neuron Kernel Interface \(NKI\)](#).
- Jonathan Bentz. 2025. [Simplify gpu programming with nvidia cuda tile in python](#).
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*.
- Michael Han Daniel Han and Unsloth team. 2023. [Unsloth](#).
- Tri Dao. 2024. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, volume 2024, pages 35549–35562.
- Yaoyao Ding, Bohan Hou, Xiao Zhang, Allan Lin, Tianqi Chen, Cody Hao Yu, Yida Wang, and Genady Pekhimenko. 2026. Tilus: A tile-level gpgpu programming language for low-precision computation. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 281–297.

864	Yue Guan, Hongtao Yu, Peng Chen, Daohang Shi,	graphics and computing architecture . <i>IEEE Micro</i> ,	920
865	Karthik Manivannan, Nicholas J Riasanovsky, Man-	28(2):39–55.	921
866	man Ren, Lei Wang, Shane Nay, Partha Kanuparth,		
867	and 1 others. 2026. Tlx: Hardware-native, evolv-	John Nickolls, Ian Buck, Michael Garland, and Kevin	922
868	able mimw gpu compiler for large-scale production	Skadron. 2008. Scalable parallel programming with	923
869	environments. <i>arXiv preprint arXiv:2605.10905</i> .	CUDA .	924
870	Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. De-	NVIDIA. NVIDIA Nsight compute .	925
871	noising diffusion probabilistic models. <i>Advances</i>		
872	<i>in neural information processing systems</i> , 33:6840–	NVIDIA. 2024a. cuBLAS library .	926
873	6851.		
874	Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan	NVIDIA. 2024b. CUTLASS: CUDA templates for lin-	927
875	Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam	ear algebra subroutines .	928
876	Sahni, Haowen Ning, and Yanning Chen. 2024. Liger	NVIDIA. 2025. ComputeEval: Evaluating large lan-	929
877	kernel: Efficient triton kernels for llm training. <i>arXiv</i>	guage models for cuda code generation .	930
878	<i>preprint arXiv:2410.10989</i> .		
879	Jiawei Hu, Hong Jia, Mahbub Hassan, Lina Yao, Brano	NVIDIA. 2026. PTX writer’s guide to interoperability .	931
880	Kusy, and Wen Hu. 2024. Lightllm: A versatile large		
881	language model for predictive light sensing . <i>Preprint</i> ,	OpenAI. 2026. Introducing GPT-5.5 .	932
882	arXiv:2411.15211.		
883	Albert Q. Jiang, Alexandre Sablayrolles, Antoine	Anne Ouyang, Simon Guo, Simran Arora, Alex L	933
884	Roux, Arthur Mensch, Blanche Savary, Chris	Zhang, William Hu, Christopher Ré, and Azalia	934
885	Bamford, Devendra Singh Chaplot, Diego de las	Mirhoseini. 2025. Kernelbench: Can llms write effi-	935
886	Casas, Emma Bou Hanna, Florian Bressand, Gi-	cient gpu kernels? <i>arXiv preprint arXiv:2502.10517</i> .	936
887	anna Lengyel, Guillaume Bour, Guillaume Lam-		
888	ple, L��lio Renard Lavaud, Lucile Saulnier, Marie-	Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya	937
889	Anne Lachaux, Pierre Stock, Sandeep Subramanian,	Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sas-	938
890	Sophia Yang, and 7 others. 2024. Mixtral of experts .	try, Amanda Askell, Pamela Mishkin, Jack Clark,	939
891	<i>Preprint</i> , arXiv:2401.04088.	Gretchen Krueger, and Ilya Sutskever. 2021. Learn-	940
892	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying	ing transferable visual models from natural language	941
893	Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gon-	supervision . <i>Preprint</i> , arXiv:2103.00020.	942
894	zalez, Hao Zhang, and Ion Stoica. 2023. Efficient		
895	memory management for large language model serv-	Mark Saroufim, Jiannan Wang, Bert Maher, Sahan	943
896	ing with pagedattention. In <i>Proceedings of the 29th</i>	Paliskara, Laura Wang, Shahin Sefati, and Manuel	944
897	<i>symposium on operating systems principles</i> , pages	Candales. 2025. Backendbench: An evaluation suite	945
898	611–626.	for testing how well llms and humans can write py-	946
899	LeetGPU. 2025. Challenges .	torch backends .	947
900	Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yux-	Benjamin Spector, Simran Arora, Aaryan Singhal, Ar-	948
901	uan Li, Zefan Wang, Jiacheng Huang, Haojie Wang,	jun Parthasarathy, Dan Fu, and Christopher Re. 2025.	949
902	Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong	Thunderkittens: Simple, fast, and Adorable kernels.	950
903	Sun. 2025. Tritonbench: Benchmarking large lan-	In <i>International Conference on Learning Representa-</i>	951
904	guage model capabilities for generating triton opera-	<i>tions</i> , volume 2025, pages 2097–2130.	952
905	tors . <i>Preprint</i> , arXiv:2502.14752.		
906	Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden,	Philippe Tillet, H. T. Kung, and David Cox. 2019. Tri-	953
907	Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang,	ton: an intermediate language and compiler for tiled	954
908	Chunli Fu, Yihan He, Samuel Hsia, and 1 others.	neural network computations . In <i>Proceedings of the</i>	955
909	2025. Kernelevolve: Scaling agentic kernel cod-	<i>3rd ACM SIGPLAN International Workshop on Ma-</i>	956
910	ing for heterogeneous ai accelerators at meta. <i>arXiv</i>	<i>chine Learning and Programming Languages</i> , MAPL	957
911	<i>preprint arXiv:2512.23236</i> .	2019, page 10–19, New York, NY, USA. Association	958
912	Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qi-	for Computing Machinery.	959
913	jing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou,	Triton Document. tri-	960
914	Yuan Zhang, Jingquan Wang, Sana Damani, and 1	ton.language.make_tensor_descriptor .	961
915	others. 2026. Sol-execbench: Speed-of-light bench-		
916	marking for real-world gpu kernels against hardware	Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhi-	962
917	limits. <i>arXiv preprint arXiv:2603.19173</i> .	wen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia,	963
918	Erik Lindholm, John Nickolls, Stuart Oberman, and	Jilong Xue, Fan Yang, and Zhi Yang. 2025. Tile-	964
919	John Montrym. 2008. Nvidia tesla: A unified	lang: A composable tiled programming model for ai	965
		systems . <i>Preprint</i> , arXiv:2504.17577.	966
		Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu,	967
		Julien Demouth, and Song Han. 2024. Smoothquant:	968
		Accurate and efficient post-training quantization for	969
		large language models . <i>Preprint</i> , arXiv:2211.10438.	970

- Shanli Xing, Yiyang Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, and 1 others. 2026. Flashinfer-bench: Building the virtuous cycle for ai-driven llm systems. *arXiv preprint arXiv:2601.00227*.
- Divakar Kumar Yadav, Tian Zhao, and Deepak Kumar. 2026. Evaluating cuda tile for ai workloads on hopper and blackwell gpus. *arXiv preprint arXiv:2604.23466*.
- Biao Zhang and Rico Sennrich. 2019. [Root mean square layer normalization](#). *Preprint*, arXiv:1910.07467.
- Keren Zhou, Tianle Zhong, Hao Wu, Jihyeong Lee, Yue Guan, Yufei Ding, Corbin Robeck, Yuanwei Fang, Jeff Niu, and Philippe Tillet. 2026. Proton: Towards multi-level, adaptive profiling for triton. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 493–506.
- Jiace Zhu, Wentao Chen, Qi Fan, Zhixing Ren, Junying Wu, Xing Zhe Chai, Chotiwit Rungrueangwutthinon, Yehan Ma, and An Zou. 2026. [Cudabench: Benchmarking llms for text-to-cuda generation](#). *Preprint*, arXiv:2603.02236.

994	Appendix
995	A Benchmark Information
996	A.1 Full Benchmark List

Table 1: Operator benchmark suite covering point-wise, reduction, attention, convolution, and data-layout kernels.

Operator Name	Source	Category	Explanation
flash-attention	TritonBench	Matrix Mult./Attn.	Standard prefill/dense attention.
flash-decode	TritonBench	Matrix Mult./Attn.	Flash decoding attention.
block-sparse-attn	TritonBench	Matrix Mult./Attn.	Block-sparse attention.
softmax	TritonBench	Reduction/Norm.	Standard Softmax.
rope-embedding	TritonBench	Point-wise	Positional embedding.
matmul_fp32_fp16_fp8	TritonBench	Matrix Mult./Attn.	Standard GEMM (supports FP16 & FP8).
matmul-int8	TritonBench	Matrix Mult./Attn.	Quantized GEMM.
streamk-matmul	TritonBench	Matrix Mult./Attn.	Stream-K scheduling optimization.
layernorm	TritonBench	Reduction/Norm.	Standard layer normalization.
rmsnorm	TritonBench	Reduction/Norm.	RMS normalization.
vector-addition	TritonBench	Point-wise	Binary addition.
mul2	TritonBench	Point-wise	Elementwise multiplication.
relu	TritonBench	Point-wise	Activation function.
destindex	TritonBench	Data Layout	Indexing/scatter/gather operations.
quantize-global	TritonBench	Point-wise	Packing values.
dequantize-rowwise	TritonBench	Point-wise	Unpacking values.
cross-entropy	TritonBench	Reduction/Norm.	Elementwise loss calculation.
matrix-transpose	TritonBench	Data Layout	Transpose / permute.
dropout	TritonBench	Point-wise	Random mask generation + apply.
swiglu	TritonBench	Point-wise	Complex gated activation.
kl-divergence	TritonBench	Reduction/Norm.	Divergence metric calculation.
fused-activation	TritonBench	Point-wise	Generic fused kernel container.
mean-reduction	TritonBench	Reduction/Norm.	Sum/mean reduction.
argmax	TritonBench	Reduction/Norm.	Index finding.
l2-norm	TritonBench	Reduction/Norm.	Norm calculation.
2d-conv	TritonBench	Stencil/Conv.	2D convolution.
top-k-selection	LeetGPU	Data Layout	Select the top- K largest elements from an array.
histogramming	LeetGPU	Reduction/Norm.	Count occurrences of each value into histogram bins.
linear-self-attn	LeetGPU	Matrix Mult./Attn.	Compute linear attention using kernel feature maps.
jacobi-stencil-2d	LeetGPU	Stencil/Conv.	Jacobi stencil 2D.
bitonic-sort	LeetGPU	Data Layout	Sort an array of floats in ascending order.
radix-sort	LeetGPU	Data Layout	Sort 32-bit unsigned integers in ascending order via radix sort.
batched-matmul	LeetGPU	Matrix Mult./Attn.	Batched matrix multiplication.
batch-normalization	LeetGPU	Reduction/Norm.	Batch normalization.
2d-max-pooling	LeetGPU	Stencil/Conv.	2D max pooling.
gaussian-blur	LeetGPU	Stencil/Conv.	Apply a Gaussian blur filter via 2D convolution.
1d-conv	LeetGPU	Stencil/Conv.	1D convolution.
3d-conv	LeetGPU	Stencil/Conv.	3D convolution.
matrix-copy	LeetGPU	Data Layout	Copy an $N \times N$ matrix element-wise.
reverse-array	LeetGPU	Data Layout	Reverse an array in-place.

Continued on next page

Table 1 – continued from previous page

Operator Name	Source	Category	Explanation
interleave	LeetGPU	Data Layout	Interleave two float arrays element-wise.
weight-dequant	LeetGPU	Point-wise	Dequantize a tiled weight matrix using per-tile scale factors.
moe-topk-gating	LeetGPU	Reduction/Norm.	Select the top- k experts per token and compute softmax gating.
leaky-relu	LeetGPU	Point-wise	Leaky ReLU activation.
sigmoid	LeetGPU	Point-wise	Sigmoid activation.

```

1  "gpu": "NVIDIA B200",
2  "peak_bw_GBs": 6539.4,
3  "peak_tflops": {
4    "fp64": 37,
5    "fp32": 1100,
6    "tf32": 1100,
7    "fp16": 2250,
8    "bf16": 2250,
9    "fp8_e4m3fn": 4500,
10   "fp8_e5m2": 4500,
11   "int8": 4500
12 },

```

Figure 6: B200.json Example

A.2 Operator Category

A.3 Config.yaml Example

Each operator ships with a config.yaml file declaring the case grid the harness will run (the cross product of input sizes and dtypes) and the verification tolerances applied to the generated kernel’s output against the PyTorch reference. The configuration is per-operator (not global) because different operators have meaningfully different shape-sensitivity profiles: matmuls are exercised at sizes where tensor-core paths kick in, reductions at sizes where occupancy starts saturating, and elementwise ops at sizes that span memory-bound and compute-bound regimes. The verify-tolerance defaults follow standard mixed-precision practice; per-operator overrides are used where the reference itself is non-associative across reduction order, in which case the tolerance is widened to absorb floating-point reassociation error.

A.4 B200 JSON Information

We use per-GPU dense peak compute ceilings derived from the NVIDIA HGX B200 / DGX B200 specifications, excluding 2:4 sparsity. For FP32 dot/MMA workloads, we use the TF32 Tensor Core dense ceiling because Triton tl.dot uses TF32 by default for FP32 inputs. The peak bandwidth value, 6539.4 GB/s, is measured on our B200 system rather than taken from the datasheet HBM bandwidth.

B Iterative LLM Code-Generation Prompts

This appendix documents the prompts used in the iterative LLM code-generation track described in Section 3.3. We show the iteration-0 and iteration- N prompt templates, followed by the building blocks they reference: the TileBench framework

```

1 benchmark:
2   warmup: 20
3   repeat: 100
4   use_cuda_graph: true
5   flush_l2: true
6   autotune: false
7 case_defaults:
8   M: 2048
9 case_grid:
10  N: [20480]
11  dtype: ["fp16", "fp32"]
12 metrics:
13   flops_expr: "M * N"
14   bytes_expr: "M * N * dtype_size"
15 plots:
16   - latency_ms
17   - bandwidth_GBs
18   - speedup

```

Figure 7: argmax (argmax.yaml) Example

guide, the per-backend API references, the operator problem description, the PyTorch reference implementation, and the strict output-format instructions. The operator’s config.yaml is shown separately in Appendix A.3. Throughout this section we use vector_add as the running example operator; for every other operator the same skeleton is filled in with the corresponding operator-specific files.

B.1 Iteration 0 Prompt Template

The iteration-0 prompt is assembled by concatenating the components below in the order shown. The model receives no prior context other than the system prompt; everything operator-specific is fed as the user message.

Iteration 0 Prompt Template

```

1 # Task: implement operator `op` for
2   ↳ TileBench
3 Follow the framework conventions below.
4   ↳ Then implement the operator
5 described. You will emit `impl_triton.py`,
6   ↳ `impl_cutile.py`.
7 ---
8 # TileBench Framework Conventions
9 <<< framework guide -- Cref{appsubsec:
10   ↳ framework_guide} >>>
11 ---
12 # Triton (triton / triton.language) API
13   ↳ Reference
14 <<< Triton API reference -- Cref{
15   ↳ appsubsec:api_reference} >>>

```

Category	#Ops	Representative operators
Point-wise	12	vector_add, relu, swiglu, weight_dequant
Reduction/Normalization	11	softmax, layernorm, argmax, moe_topk_gating
Matrix Multiplication/Attention	8	matmul, matmul_int8, flash_attention, linear_self_attention
Stencil/Convolution	6	1d_conv, 3d_conv, gaussian_blur, jacobi_stencil_2d
Data Layout	8	matrix_copy, matrix_transpose, bitonic_sort, radix_sort
Total	45	

Table 2: TileBench operator categories. TileBench contains 45 operators: 26 adapted from TritonBench and 19 adapted from LeetGPU.

```

16 ---
17
18 # cuTile (cuda.tile) API Reference
19 <<< cuTile API reference -- Cref{
    ↳ appsubsec:api_reference} >>>
20
21 ---
22
23 # Operator description: `<op>`
24 <<< natural-language problem
    ↳ description --
25     Cref{appsubsec:problem_description}
    ↳ >>>
26
27 ---
28
29 # `config.yaml` for `<op>`
30 <<< config.yaml -- Cref{appsubsec:
    ↳ config.yaml_example} >>>
31
32 ---
33
34 # PyTorch reference: `impl_torch.py`
35 <<< impl_torch.py -- Cref{appsubsec:
    ↳ torch_reference} >>>
36
37 ---
38
39 <<< output-format instructions --
40     Cref{appsubsec:output_format} >>>

```

Iteration N Prompt Template ($N > 0$)

```

1 # Task: improve operator `<op>` for
    ↳ TileBench (iteration N)
2
3 This is iteration N of the refinement
    ↳ loop. Your previous
4 iteration's results are reported below.
    ↳ Read the trajectory
5 carefully -- if your last iteration
    ↳ regressed vs the best
6 verify-clean iter so far, consider
    ↳ going back to that approach
7 as your starting point and trying a
    ↳ different optimization.
8 Then re-emit `impl_triton.py`, `
    ↳ impl_cutile.py`.
9
10 ## Iteration trajectory so far
11
12 (One row per iteration; each row
    ↳ reports cfg, stop_score,
13 speedup_vs_torch, and verify status
    ↳ for triton and cutile.
14 Shown schematically as two per-backend
    ↳ tables for readability;
15 the actual prompt presents both
    ↳ backends as a single wide row.)
16
17 triton:
18   iter 0  cfg={...}  score=42.3%
    ↳ speedup=0.85x  verify=ok
19   iter 1  cfg={...}  score=61.0%
    ↳ speedup=1.23x  verify=ok
20   ...
21
22 cutile:
23   iter 0  cfg={...}  score=17.1%
    ↳ speedup=0.32x  verify=ok
24   iter 1  cfg={...}  score=24.5%
    ↳ speedup=0.47x  verify=fail (2
    ↳ cases)
25   ...
26
27 `cfg` is the configuration the kernel
    ↳ actually used, returned by
28 `get_last_config()`. `speedup_vs_torch`
    ↳ is `torch_latency` /
29 kernel_latency` -- values > 1 mean the
    ↳ kernel beat the PyTorch
30 reference; < 1 means torch is still
    ↳ faster.
31

```

B.2 Iteration N Prompt Template ($N > 0$)

For iterations after iteration 0, the prompt is rebuilt from scratch each iteration rather than appended to a running chat. It still ends with the framework guide, API references, problem description, config.yaml, PyTorch reference, and output-format instructions (identical to iteration 0), but is prefixed with a trajectory table, the previous iteration's source files, and, when the previous iteration regressed or failed verification, the best verify-clean source files seen so far as a rebuild reference.

```

32 Best verify-clean `triton` so far: iter
    ↳ K with stop_score=X.X%.
33 Best verify-clean `cutile` so far: iter
    ↳ K with stop_score=X.X%.
34 (Regression warnings are appended here
    ↳ when applicable.)
35
36 ## Feedback from iteration N-1
37
38 (structured feedback: compile errors
    ↳ per backend, per-case timeout
39 errors per backend, verification
    ↳ failures, per-backend
    ↳ stop_score
40 and report_mean, and a worst-first per
    ↳ -(backend, dtype, case)
41 roofline table sorted by roofline_pct)
42
43 ---
44
45 ## Your previous `impl_triton.py` (
    ↳ iteration N-1)
46
47 ```python
48 <previous impl_triton.py source>
49 ```
50
51 ## Your previous `impl_cutile.py` (
    ↳ iteration N-1)
52
53 ```python
54 <previous impl_cutile.py source>
55 ```
56
57 ---
58
59 ## Best verify-clean code so far
60 (emitted only for backends whose
    ↳ previous iteration regressed
61 or broke verification, AND whose best-
    ↳ so-far iteration is not
62 the previous iteration)
63
64 ### Best-so-far `impl_<backend>.py` (
    ↳ iter K, stop_score=X.X%)
65
66 Your previous iteration regressed (or
    ↳ broke verify) for this backend.
67 Use this code as the starting point and
    ↳ apply a different optimization.
    ↳
68
69 ```python
70 <best verify-clean impl_<backend>.py
    ↳ source>
71 ```
72
73 ---
74
75 # Reminders: TileBench Framework
    ↳ Conventions
76 <<< framework guide -- Cref{appsubsec:
    ↳ framework_guide} >>>
77
78 ---
79
80 # Triton (triton / triton.language) API
    ↳ Reference

```

```

81 <<< Triton API reference -- Cref{
    ↳ appsubsec:api_reference} >>>
82
83 # cuTile (cuda.tile) API Reference
84 <<< cuTile API reference -- Cref{
    ↳ appsubsec:api_reference} >>>
85
86 # Operator description: `<op>`
87 <<< problem description -- Cref{
    ↳ appsubsec:problem_description}
    ↳ >>>
88
89 ---
90
91 # `config.yaml` for `<op>`
92 <<< config.yaml -- Cref{appsubsec:
    ↳ config.yaml_example} >>>
93
94 ---
95
96 # PyTorch reference: `impl_torch.py`
97 <<< impl_torch.py -- Cref{appsubsec:
    ↳ torch_reference} >>>
98
99 ---
100
101 <<< output-format instructions --
    ↳ Cref{appsubsec:output_format} >>>
102

```

B.3 TileBench Framework Guide

The framework guide specifies the file structure each implementation must produce, the `run(*args, **kwargs)` and `get_last_config()` contract, the forbidden patterns (triton.autotune, cutileAutotuner, delegation to PyTorch / cuDNN / cuBLAS, output caching), recommended tile-size bounds for B200, multi-dtype handling, and a pre-return checklist. An abridged transcription is shown below: markdown emphasis and decorative emoji from the source file have been stripped, but section headings, rules, code templates, and numeric bounds are reproduced as in the original.

TileBench Framework Guide (framework_guide.md)

```

1 # TileBench Framework Guide
2
3 Read this carefully. Your generated
    ↳ kernels MUST follow this
    ↳ contract
4 or the benchmark harness will fail to
    ↳ load / run / evaluate them.
5
6 ## Files you must produce
7
8 For operator `<op>`, write exactly two
    ↳ files:
9

```

```

10 1. `impl_triton.py` -- Triton kernel +
    ↳ Python `run()` wrapper
11 2. `impl_cutile.py` -- cuTile kernel +
    ↳ Python `run()` wrapper
12
13 You do NOT generate `impl_torch.py` --
    ↳ it is copied from the
14 human-written reference implementation.
    ↳ Your `run()` function's
15 signature MUST match `impl_torch.run()`
    ↳ exactly (same positional
16 args, same keyword args).
17
18 ## Required exports (both impls)
19
20 def run(*args, **kwargs):
21     """Execute the operator. Must
    ↳ produce a single torch.Tensor (
    ↳ or
22     tuple of tensors) that bit-
    ↳ equivalently matches
23     impl_torch.run() within the
    ↳ tolerances in config.yaml's
24     `verify:` section."""
25
26 def get_last_config() -> dict |
    ↳ None:
27     """Return the configuration
    ↳ this run() actually used, as a
28     flat dict (e.g. {'BLOCK_M': 128
    ↳ , 'BLOCK_N': 64,
29     'num_warps': 4, 'num_stages': 2
    ↳ }). The harness logs this so
30     the iterative-refinement loop
    ↳ can show the trajectory of
31     configurations tried."""
32
33 ## No autotune -- you pick one
    ↳ configuration per iteration
34
35 The harness does NOT call autotune. The
    ↳ kernel runs with a single
36 hard-coded configuration that YOU
    ↳ choose. The 10-iteration
    ↳ refinement
37 loop is the search mechanism: at each
    ↳ iteration the model sees the
38 previous iteration's (config ->
    ↳ roofline_pct, latency_ms,
39 speedup_vs_torch), and proposes the
    ↳ next configuration.
40
41 The following are FORBIDDEN in
    ↳ generated code and will be
    ↳ rejected
42 at load:
43
44 - `triton.autotune(...)` decorator
45 - `from core.cutile_autotune import
    ↳ CutileAutotuner`
46 - `ct.tune.exhaustive_search` / `
    ↳ ct_experimental.autotune_launch
    ↳ `
47 - Any other in-kernel autotune-style
    ↳ search
48
49 ## Triton template (no autotune)
50

```

```

51 import torch
52 import triton
53 import triton.language as tl
54
55 _LAST_CFG: dict = {}
56
57 @triton.jit
58 def _op_kernel(x_ptr, out_ptr,
    ↳ n_elements,
59             BLOCK_SIZE: tl.
    ↳ constexpr):
60     pid = tl.program_id(0)
61     offs = pid * BLOCK_SIZE + tl.
    ↳ arange(0, BLOCK_SIZE)
62     mask = offs < n_elements
63     x = tl.load(x_ptr + offs, mask=
    ↳ mask)
64     tl.store(out_ptr + offs, x,
    ↳ mask=mask)
65
66 def run(x):
67     output = torch.empty_like(x)
68     n_elements = x.numel()
69
70     BLOCK_SIZE = 2048
71     num_warps = 4
72     num_stages = 2
73
74     grid = (triton.cdiv(n_elements,
    ↳ BLOCK_SIZE),)
75     _op_kernel[grid](
76         x, output, n_elements,
77         BLOCK_SIZE=BLOCK_SIZE,
78         num_warps=num_warps,
79         num_stages=num_stages,
80     )
81
82     _LAST_CFG.clear()
83     _LAST_CFG.update({"BLOCK_SIZE":
    ↳ BLOCK_SIZE,
84                     "num_warps":
    ↳ num_warps,
85                     "num_stages":
    ↳ num_stages})
86     return output
87
88 def get_last_config() -> dict |
    ↳ None:
89     return dict(_LAST_CFG) if
    ↳ _LAST_CFG else None
90
91 Rules:
92 - `_LAST_CFG` is a module-level `dict`
    ↳ -- mutate with `.clear()` +
93     `.update()`, never use the `global`
    ↳ keyword.
94 - Record EVERY parameter that
    ↳ influenced the kernel launch (
    ↳ block
95     sizes, `num_warps`, `num_stages`,
    ↳ group size for grouped-launch
96     matmuls, etc.) so the iterative-
    ↳ refinement loop can see exactly
97     what was tried.
98
99 ## cuTile template (no autotune)
100
101 import torch

```

```

102 import cuda.tile as ct
103 import numpy as np
104
105 ConstInt = ct.Constant[int]
106
107 _LAST_CFG: dict = {}
108
109 @ct.kernel
110 def _op_kernel(x, output, TILE:
    ↳ ConstInt):
111     bid = ct.bid(0)
112     x_tile = ct.load(x, index=(bid
    ↳ ),, shape=(TILE,)),
113     padding_mode=
    ↳ ct.PaddingMode.ZERO)
114     ct.store(output, index=(bid,),
    ↳ tile=x_tile)
115
116 def run(x):
117     output = torch.empty_like(x)
118     n_elements = x.numel()
119     stream = torch.cuda.
    ↳ current_stream()
120
121     TILE = 2048
122     occupancy = 8
123
124     grid = (ct.cdiv(n_elements,
    ↳ TILE), 1, 1)
125     kernel = _op_kernel.with_hints(
    ↳ occupancy=occupancy)
126     ct.launch(stream, grid, kernel,
    ↳ (x, output, TILE))
127
128     _LAST_CFG.clear()
129     _LAST_CFG.update({"TILE": TILE,
    ↳ "occupancy": occupancy})
130     return output
131
132 def get_last_config() -> dict |
    ↳ None:
133     return dict(_LAST_CFG) if
    ↳ _LAST_CFG else None
134
135 Rules:
136 - `_LAST_CFG` is a module-level dict --
    ↳ mutate with `.clear()` +
137   `.update()`, never use the `global`
    ↳ keyword.
138 - cuTile tile-shape dimensions MUST be
    ↳ powers of 2.
139 - For non-power-of-2 problem dims: use
    ↳ `padding_mode=
140   ct.PaddingMode.ZERO` on `ct.load`,
    ↳ and `ct.store` silently
141   drops OOB writes.
142 - For max-reduction kernels needing to
    ↳ neutralize OOB: use
143   `padding_mode=ct.PaddingMode.NEG_INF`.
    ↳
144 - `ct.store()` only accepts static
    ↳ indices. For runtime-computed
145   scatter indices, use `ct.scatter()`.
146
147 ## Choosing tile / BLOCK sizes
148
149 Lower bounds (do NOT pick below these;
    ↳ all values powers of 2):

```

```

150
151 1D elementwise / pointwise      :
    ↳ BLOCK / tile >= 512
152 Row-reduction / softmax / norm :
    ↳ reduce dim >= 256
153 Stencil / 2D conv              :
    ↳ spatial dim >= 256
154 Matmul / attention (TC)        :
    ↳ BLOCK_M >= 64, BLOCK_K >= 32
155
156 Upper bounds (do NOT pick above these;
    ↳ B200 sm_100, 228 KB shmem,
157 64 K registers per CTA):
158
159 1D pointwise : BLOCK_SIZE <= 8192
    ↳ fp16/fp32;
160                                BLOCK_SIZE <= 16384
    ↳ int8
161 Matmul tile : BLOCK_M x BLOCK_N <=
    ↳ 256x256 fp16,
162                                <=
    ↳ 128x256 fp32,
163                                <=
    ↳ 256x256 fp8/int8
164 Matmul K-tile : BLOCK_K <= 128 fp16/
    ↳ fp8/int8,
165                                BLOCK_K <= 64 fp32
166
167 Sensible starting points for iter 0:
168
169 1D pointwise (Triton)          :
    ↳ BLOCK_SIZE=2048, warps=4,
    ↳ stages=2
170 1D pointwise (cuTile)          :
    ↳ tile=2048, occupancy=8
171 Row reduction (Triton, 1 CTA/row):
    ↳ BLOCK_N=1024, warps=4
172 Matmul fp16/bf16 (Triton)      : 12
    ↳ 8x128x64, warps=8, stages=3
173 Matmul fp16/bf16 (cuTile)      : tm
    ↳ =128, tn=128, tk=64, occupancy=
    ↳ 8
174 Matmul fp32 (Triton)           : 12
    ↳ 8x64x32, warps=4, stages=2
175
176 ## Multi-dtype support
177
178 If `config.yaml`'s `case_grid.dtype`
    ↳ lists multiple dtypes (e.g.
179 ["fp16", "bf16", "fp32"]), the `run()`
    ↳ function must work for ALL
180 of them in a single `run()` call with
    ↳ ONE configuration shared
181 across all dtypes. Use a single dtype-
    ↳ polymorphic kernel:
182
183 - Triton's `tl.dot` and cuTile's `ct.
    ↳ mma` adapt to input dtype
184 automatically.
185 - Do NOT branch `BLOCK_M`, `num_warps`,
    ↳ etc. on `a.dtype`; pick
186 one tile that is correct for ALL
    ↳ dtypes in the grid.
187
188 Picking a tile that works for every
    ↳ dtype: the binding constraint
189 is usually fp32 (4 bytes/element x
    ↳ stages x tile area must fit in

```

```

190 228 KB shmem on B200). If the fp16 tile
    ↳ would not fit in fp32,
191 shrink the tile so it fits fp32 -- then
    ↳ fp16/bf16 just use spare
192 capacity.
193
194 ## Hardware constraints to remember (B2
    ↳ 00, sm_100)
195
196 - TMA via `tl.make_tensor_descriptor`:
    ↳ the descriptor's
197 `block_shape` last dim must occupy >=
    ↳ 16 bytes (>= 8 fp16,
198 >= 4 fp32, >= 16 int8). For 1D
    ↳ pointwise ops, do NOT use TMA
199 descriptors -- use plain `tl.load`.
200 - Tensor Core: `tl.dot` / `ct.mma`
    ↳ triggers MMA hardware.
201 Triton's default `input_precision="tf
    ↳ 32"` promotes fp32 inputs
202 to TF32 TC (1100 TFLOPS dense ceiling
    ↳ ).
203 - Powers of two: cuTile tile dims must
    ↳ be powers of 2; non-pow2
204 cases use padding.
205
206 ## FORBIDDEN: delegating computation to
    ↳ PyTorch / cuDNN / cuBLAS
207
208 The `run()` function in `impl_triton.py
    ↳ ` MUST perform the
209 operator's computation through the `
    ↳ @triton.jit` kernel, and
210 `run()` in `impl_cutile.py` MUST
    ↳ perform it through the
211 `@ct.kernel`. The following are NOT
    ↳ allowed inside `run()`:
212
213 - `torch.nn.functional.*` (
    ↳ scaled_dot_product_attention,
    ↳ softmax,
214 layer_norm, linear, conv*,
    ↳ cross_entropy, batch_norm,
215 dropout, etc.)
216 - `torch.matmul`, `torch.mm`, `torch.
    ↳ bmm`, `torch.einsum`,
217 `torch.linalg.*`
218 - `torch.softmax`, `torch.argmax`, `
    ↳ torch.sort`, `torch.topk`,
219 `torch.bincount`, `torch.flip`, `
    ↳ torch.transpose` as a
220 substitute for the actual operator
    ↳ computation
221 - Any function whose name on its own is
    ↳ a literal description of
222 the operator (e.g. `torch.norm`
    ↳ inside `l2_norm`).
223 - Importing the reference (`from
    ↳ impl_torch import run`).
224
225 A "decoy" kernel that is defined but
    ↳ not invoked on the real
226 workload, or invoked only on a one-
    ↳ element input, will be
227 detected and rejected.
228
229 Valid uses of `torch.*` inside `run()`
    ↳ are limited to: tensor

```

```

230 allocation (`torch.empty`, `torch.
    ↳ empty_like`, `torch.zeros`),
231 shape manipulation that does not
    ↳ compute (`contiguous()`,
232 `.view()`, `.reshape()`, `.transpose()
    ↳ `, `.unsqueeze()`,
233 `.permute()`) -- but not as a substitute
    ↳ for the operator),
234 stream / event management, and dtype-
    ↳ only casts on metadata
235 (not on the data path).
236
237 ## FORBIDDEN: caching outputs across `
    ↳ run()` calls
238
239 Do NOT add a Python-level cache that
    ↳ returns a previously-computed
240 output when the inputs look identical.
    ↳ The harness passes the
241 same input objects (or freshly-
    ↳ regenerated tensors with the
    ↳ same
242 values) to `run()` many times during
    ↳ warmup, verify, and timing.
243 The harness detects this (it
    ↳ regenerates fresh inputs
    ↳ between
244 warmup and the timed loop) and flags
    ↳ any implementation whose
245 measured throughput exceeds the
    ↳ hardware roofline as a cache
246 cheat.
247
248 ## Common pitfalls (the harness rejects
    ↳ these)
249
250 1. Importing modules not listed: stay
    ↳ within `torch`, `triton`,
251 `triton.language`, `cuda.tile`, `
    ↳ cuda.tile_experimental`
252 (optional), `math`, `numpy as np`.
    ↳ Do NOT import
253 `core.cutile_autotune`.
254 2. Wrong `run()` signature: if `
    ↳ impl_torch.run(x, y, BATCH, M,
255 N, K)` takes 6 positional args, `run
    ↳ ()` must take the same 6.
256 3. Returning a list when impl_torch
    ↳ returns a tensor.
257 4. Not handling OOB tail tiles: input
    ↳ sizes from `config.yaml`
258 may not be exact multiples of
    ↳ BLOCK_SIZE. Always mask.
259 5. Mutating inputs in-place: the
    ↳ harness re-uses input tensors
260 across the verify + timing
    ↳ repetitions. Clone if needed.
261 6. `get_last_config()` returning wrong
    ↳ type: must return dict or
262 None. Do not return Config objects
    ↳ or SimpleNamespace.
263
264 ## What the engine does with generated
    ↳ code
265
266 For each `(case, dtype)` from `config.
    ↳ yaml`'s `case_grid`:

```

```

267 1. Generates inputs via `data/tensors`.
    ↳ py's `GENERATORS[<op>]`.
268 2. Calls `impl_torch.run(*inputs)` and
    ↳ times it via NVIDIA
269 Proton (the PyTorch baseline for `
    ↳ speedup_vs_torch`).
270 3. Calls `impl_triton.run(*inputs)`
    ↳ once for verify; checks
271 output matches the torch reference
    ↳ within `verify:`
272 tolerances. On failure, no timing is
    ↳ collected for that
273 backend.
274 4. If verify passes: times `impl_triton.
    ↳ run(*inputs)` over a
275 rotating sequence of fresh inputs, 2
    ↳ 0 warmup + 100 measured
276 iterations, inside a `proton.scope("
    ↳ launch")`. Mean
277 latency from the Proton hatchet tree
    ↳ is reported.
278 5. Same flow for `impl_cutile.run()`.
279 6. `get_last_config()` is called once
    ↳ per backend per case and
280 the returned dict is included in the
    ↳ next iteration's
281 feedback.
282
283 The refinement loop runs the full
    ↳ iteration budget (default 10
284 iters). There is no roofline-based
    ↳ early stopping; both Triton
285 and cuTile are regenerated every
    ↳ iteration. After the budget is
286 exhausted, the best verify-clean
    ↳ iteration per backend is
287 promoted as the final result. Each
    ↳ iteration's `stop_score`
288 (capped `roofline_pct` on the single
    ↳ largest case per dtype,
289 averaged across the supported dtypes)
    ↳ is reported as feedback
290 so the next iteration can see how close
    ↳ to peak the previous
291 attempt got.
292
293 ## TL;DR checklist before returning
    ↳ code
294
295 - [ ] Two files: `impl_triton.py` and `
    ↳ impl_cutile.py`
296 - [ ] Each exports `run(...)` and
    ↳ `get_last_config() -> dict | None`
297
298 - [ ] `run()` signature matches `
    ↳ impl_torch.run()` exactly
299 - [ ] NO `triton.autotune`, NO `
    ↳ CutileAutotuner` import
300 - [ ] One hard-coded configuration,
    ↳ recorded in `_LAST_CFG`
301 - [ ] No `global` keyword for `
    ↳ _LAST_CFG`
302 - [ ] Multi-dtype handled in a single `
    ↳ run()`
303 - [ ] OOB / non-pow-2 handled with
    ↳ masks (Triton) or
304 `padding_mode` (cuTile)
305 - [ ] No in-place mutation of inputs

```

B.4 Triton and cuTile API References

For each active backend, the prompt prepends an authoritative API reference that scopes the model to the exact library versions used by the harness (Triton 3.6.0 and the cuTile pip packages shipped with TileBench). Each reference is a long document; reproducing either in full would dominate the appendix, so we describe their structure and show a representative excerpt.

Structure. Each API reference is organized as numbered topical sections that, at a high level, cover (i) import boilerplate and data types, (ii) the kernel programming model and tile creation, (iii) memory operations, (iv) compute primitives (arithmetic, reductions, matrix multiply), (v) control flow and synchronization, (vi) compiler hints, (vii) the autotune pattern (explicitly disabled in this pipeline; see the preamble below), (viii) the TileBench `run()` / `get_last_config()` contract and common kernel patterns, and (ix) critical constraints, gotchas, and a quick-reference cheatsheet. The exact section list differs between the two references because Triton and cuTile expose different abstractions.

Preamble (Triton). Each reference is prefixed by a short preamble that anchors the model to the installed library version and explicitly invalidates the reference's autotune content (since the LLM track forbids autotune). The Triton preamble is:

Triton API Reference Preamble

- 1 Treat the API reference below as
 - ↳ authoritative for the Triton
 - ↳ version
- 2 installed in this repo (3.6.0); do NOT
 - ↳ use APIs from later versions
- 3 you may have seen in training data.
 - ↳ Every ``tl.*`` / ``triton.*``
 - ↳ symbol
- 4 you use must appear in this reference.
- 5
- 6 IMPORTANT -- autotune sections of this
 - ↳ reference do NOT apply. Use
- 7 the reference for kernel-body syntax (
 - ↳ `tl.load`, `tl.store`, `tl.dot`,
- 8 `masking`, `make_block_ptr`,
 - ↳ `make_tensor_descriptor`, etc.);
 - ↳ ignore
- 9 everything about `cfg search` / `autotune`
 - ↳ wrappers.

The cuTile preamble is analogous.

Section excerpt. As a concrete example of the section-level granularity, the cuTile reference's memory-operations section opens as shown below. Each entry gives the function signature, a parameter list, and a few representative call examples; per-section content varies in length but follows this shape.

cuTile API Reference: §6 Memory Operations (Excerpt)

```
1 ## 6. Memory Operations
2
3 ### 6.1 ct.load()
4
5 Signature:
6     tile = ct.load(array, index, shape,
7         ↳ *,
8             order="C",
9             padding_mode=
10         ↳ PaddingMode.UNDETERMINED,
11             latency=None,
12             allow_tma=None)
13
14 Parameters:
15     array      : Array
16                 Source array (torch
17                 ↳ tensor)
18     index      : tuple[int | Tile,
19                 ↳ ...]
20                 Tile-space
21                 ↳ coordinates
22     shape      : tuple[const int,
23                 ↳ ...]
24                 Tile shape (all dims
25                 ↳ must be power of 2);
26                 ↳ () for 0D scalar
27     tile       : "C" / "F" / tuple
28                 Axis mapping: "C"=
29                 ↳ identity, "F"=reversed,
30                 ↳ tuple=explicit
31                 ↳ permutation
32     padding_mode : PaddingMode
33                 OOB element handling
34     latency     : const int
35                 DRAM traffic hint, 1
36                 ↳ (low) to 10 (high)
37     allow_tma   : const bool
38                 Enable/disable TMA (
39                 ↳ default True)
40
41 Examples:
42     tile = ct.load(x, index=(bid,),
43         ↳ shape=(TILE,)) # 1D
44     tile = ct.load(x, index=(row, j),
45         ↳ shape=(1, TILE),
46                 padding_mode=ct.
47         ↳ PaddingMode.ZERO) #
48         ↳ 2D padded
49     tile = ct.load(Q, index=(b, h, i,
50         ↳ 0), shape=(1,1,M,D)) # 4D
51     tile = ct.load(K, index=(b, h, 0,
52         ↳ j), shape=(1,1,D,N),
```

```

36         order=(0,1,3,2))
        ↳ #
        ↳ transposed
37     scalar = ct.load(x, index=(i,),
        ↳ shape=())
        ↳ 0D scalar
38
39     """ 6.2 ct.store()
40
41     Signature:
42         ct.store(array, index, tile, *,
43                 order="C",
44                 latency=None,
45                 allow_tma=None)
46
47     Parameters:
48         array      : Array
49                     Destination array
50         index      : tuple[int | Tile, ...]
51                     Tile-space coordinates
52         tile       : Tile or scalar
53                     Data to store
54         order      : "C" / "F" / tuple
55                     Axis mapping
56         latency    : const int
57                     Latency hint
58         allow_tma  : const bool
59                     Enable/disable TMA
60
61     OOB writes are silently ignored (no-op).
        ↳

```

```

15         output[i] = x[i] + y[i]
16
17     where i ranges from 0 to n-1.
18
19     """
20     ## Implementation Requirements
21
22     - Use only native features (external
        ↳ libraries are not permitted)
23     - The `solve` function signature must
        ↳ remain unchanged
24     - The final result must be stored in
        ↳ the array `output`
25
26     ## Example 1
27
28     Input: x = [1, 2, 3, 4], y = [5, 6
        ↳ , 7, 8]
29     Output: [6, 8, 10, 12]
30
31     ## Example 2
32
33     Input: x = [-1.5, 2.0, 0.0], y = [
        ↳ 0.5, -2.0, 3.0]
34     Output: [-1.0, 0.0, 3.0]
35
36     ## Constraints
37
38     - 1 <= n
39     - `x` and `y` have the same length

```

B.5 Operator Problem Description

Each operator has a natural-language problem description at benchmarks/problems/current/<op>_current.md that gives the mathematical definition, the input contract, the output contract, and a small worked example. Below we show the file for the vector_add operator as a representative example.

Problem (vector_add_current.md)

```

1  # Vector Addition
2
3  Implement a program that performs
    ↳ element-wise addition of two 1D
4  arrays of the same length.
5
6  The input consists of two arrays:
7
8  - `x`: A 1D array.
9  - `y`: A 1D array of the same length as
    ↳ `x`.
10
11 The output should be written to the `
    ↳ output` array, which has the
12 same length as the inputs.
13
14 The operation is defined mathematically
    ↳ as:

```

B.6 PyTorch Reference Implementation

The PyTorch reference (impl_torch.py) is included verbatim in the prompt. It serves both as a precise specification of the operator's input/output contract and as the correctness oracle the generated kernels are validated against. For vector_add the reference is:

PyTorch (vector_add/impl_torch.py)

```

1 def run(x, y):
2     return x + y

```

The harness checks that the generated kernels' run() has the same positional signature as the reference, and that its output matches the reference within the tolerances declared in config.yaml's verify: block (or per-dtype defaults when absent).

B.7 Output Format Instructions

The final block of every prompt enforces a strict, machine-parseable output format. Generated text is parsed by a regular expression that matches "python title="impl_<backend>.py" fenced code blocks; anything outside the requested blocks

is discarded. Both Triton and cuTile are emitted at every iteration, and the instructions is the following:

Output Format Instructions

```

1  ## Output format (STRICT)
2
3  Return EXACTLY 2 fenced code blocks, in
   ↳ this order, with these
4  titles:
5
6  ```python title="impl_triton.py"
7  # full Python file content here
8  ```
9
10 ```python title="impl_cutile.py"
11 # full Python file content here
12 ```
13
14 Emit ONLY the `impl_triton.py`, `
   ↳ impl_cutile.py` files. No other
15 code blocks. No prose between or after
   ↳ the blocks beyond a 1-2
16 sentence summary of your approach. Do
   ↳ NOT include test code, do
17 NOT include PyTorch reference code (
   ↳ that file is provided by the
18 framework).
```

C Additional Evaluation Metrics

C.1 Evaluation Modes and Aggregation Conventions

Per-case timing. For operator o , backend b , dtype d , and input case c , the benchmark records latency $T_{o,b,d,c}$. This is the arithmetic mean of 100 timed launches following 20 warmup launches. CUDA graph replay is enabled, and the L2 cache is evicted before each measured launch outside the timed Proton scope. The complete operator run() is timed as one unit, so a multi-kernel implementation retains any intended producer-to-consumer cache reuse inside the operator boundary.

Geometric-mean aggregation. Every paper-level aggregate above the per-case latency uses a geometric mean. The per-case speedup over PyTorch is

$$S_{o,b,d,c} = \frac{T_{o,\text{torch},d,c}}{T_{o,b,d,c}}. \quad (1)$$

For the complete sweep of operator o , we assign equal weight to every valid dtype and case pair:

$$S_{o,b} = \text{GM}_{(d,c) \in \mathcal{V}_o} S_{o,b,d,c}, \quad (2)$$

where $\mathcal{V}_o$ is the set of valid cases for operator o . The suite-level result gives equal weight to each of

the 45 operators:

$$S_b^{\text{suite}} = \text{GM}_{o \in \mathcal{O}} S_{o,b}. \quad (3)$$

Category results use the same construction over operators in the category. The direct operator-level Triton and cuTile ratio is

$$Q_o = \frac{S_{o,\text{Triton}}}{S_{o,\text{cuTile}}}. \quad (4)$$

Thus, $Q_o > 1$ favors Triton and $Q_o < 1$ favors cuTile. Because the PyTorch factor cancels case by case, Q_o is also the geometric mean of $T_{o,\text{cuTile},d,c}/T_{o,\text{Triton},d,c}$ over the complete sweep. Medians reported in RQ1 and RQ2 are distribution statistics over the 45 operator-level values and are not used as averages.

Roofline aggregation and tuning gain. Let $R_{o,b,d,c}^m$ denote the roofline utilization of mode $m \in \{\text{default}, \text{autotuned}\}$. The operator-level value plotted in the paper is

$$\bar{R}_{o,b}^m = \text{GM}_{(d,c) \in \mathcal{V}_o} R_{o,b,d,c}^m. \quad (5)$$

The operator-level autotuning gain is

$$G_{o,b} = \text{GM}_{(d,c) \in \mathcal{V}_o} \frac{R_{o,b,d,c}^{\text{autotuned}}}{R_{o,b,d,c}^{\text{default}}}, \quad (6)$$

and the suite-level gain is $\text{GM}_o G_{o,b}$. Threshold counts such as $R \geq 0.8$ are evaluated on $\bar{R}_{o,b}^{\text{autotuned}}$.

RQ3 case selection. RQ3 does not average NCU counters across the sweep. For each supported operator–dtype pair, we select the maximum case recorded in the NCU catalogue and profile the exact autotuned winner used for that case. Performance ratios remain based on the corresponding Proton/CSV latency. NCU counters explain the execution mechanism but do not replace the benchmark latency.

RQ4 aggregation. For the LLM track, the correctness-gated speedup of an executed iteration is the PyTorch-to-generated-kernel latency ratio of the representative large verify-clean case saved by the evaluation pipeline. It is a single-case ratio rather than an average over dtypes. BestSpeedup@10 is the maximum verify-clean speedup observed before the run ends or the backend freezes. TokenCost@10 sums the tokens consumed by the executed iterations for that backend; skipped iterations after freezing contribute neither tokens nor speedup. TokenEfficiency@10 divides the best speedup by token cost in millions.

C.2 NCU Profiling Protocol and Kernel Attribution

The systematic Triton and cuTile profiling set contains 220 profiles: 110 supported operator–dtype pairs for Triton and the same 110 pairs for cuTile. Each profile uses the NCU-catalogued maximum input and the same autotuned winning configuration evaluated by the main benchmark. PyTorch is profiled selectively when its internal dispatch path is needed to interpret the software-baseline comparison, for example cuDNN convolution, cuBLAS GEMM, CUB radix sort, compiled flex_attention, or a generic ATen reduction.

Latency source. All operator latency and speedup values are obtained from Proton measurements. NCU is used only for profiling and diagnosing performance phenomena, and its reported kernel duration is not used in our performance results.

Multi-kernel attribution. For a single-kernel implementation, the profile directly corresponds to the measured operator. For a multi-kernel implementation, the CSV latency covers the full kernel sequence, while counter-level observations are attributed to the stage that generates them. Examples include the two destindex scatter kernels, the five linear_self_attention launches grouped into activation, KV-GEMM, column-reduction, and output-GEMM stages, and the repeated histogram, scan, and scatter stages of radix_sort. For the suite-level shared-memory analysis in Appendix D.3.7, the primary kernel is defined as the launch with the largest GPU duration inside the profile. This rule yields one Triton and one cuTile primary kernel for every operator–dtype pair.

Evidence hierarchy. We diagnose a performance gap using the following sequence:

1. inspect the source implementation and confirm the algorithm, task decomposition, precision path, and winning configuration;
2. inspect the generated SASS and its source-correlated hotspots when the final profile exposes them;
3. compare dynamic instruction count, memory requests and sectors, TMA and MMA paths, register and shared-memory use, occupancy, pipeline activity, and stall composition;

4. relate those observations to the corresponding Proton/CSV latency and dtype trend.

Compiler IR was not materialized in the finalized artifacts used for this analysis, and no compilation was rerun during validation. We therefore do not quote TTIR, TTGIR, or tile IR; all source-to-lowering claims are supported at the SASS level. Opcode-family counts are extracted from NCU source tables. When source correlation undercounts a crowded line, the counter-level `smsp__inst_executed.sum` total remains authoritative and the per-family executed count is treated as a lower bound.

A stall metric alone is not treated as causal evidence. For example, a long-scoreboard sample on an MMA line may indicate that the MMA is waiting for a preceding gather or staging chain rather than that Tensor-Core arithmetic is intrinsically slow. We therefore interpret stalls together with instruction mix, dependency producers, resource use, memory transactions, and measured latency.

Scope of the diagnosis. The profiles describe the current source implementations, compiler versions, autotuned winners, and B200 architecture. They support recurring mechanisms across the suite, but they do not imply that every occurrence of a given source primitive must produce the same hardware path. The generated instruction sequence remains dependent on dtype, tile shape, layout, and compiler decisions.

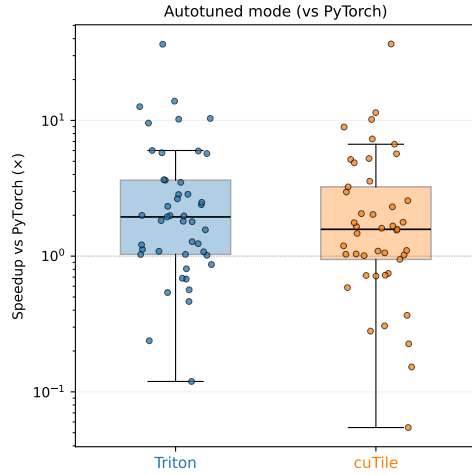
C.3 PyTorch Baseline

PyTorch serves as the semantic reference, correctness oracle, and practical software baseline for reporting speedup. Its execution path is operator dependent and may involve eager tensor operations, generic ATen kernels, compiled kernels, or optimized vendor libraries. Therefore, speedup over PyTorch reflects the practical benefit of a custom kernel over the available software path, while RQ3 focuses primarily on explaining the performance differences between the matched Triton and cuTile implementations.

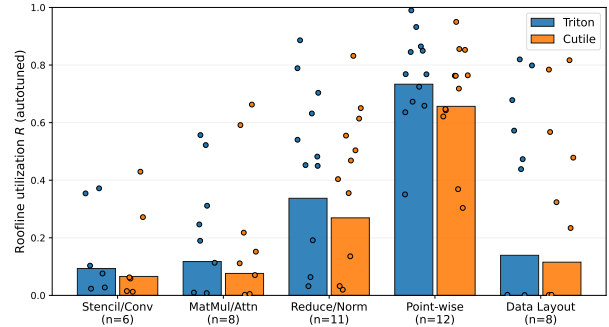
D Additional Evaluation Results

D.1 RQ1: Autotuned Speedup Distributions and Category Breakdowns

Figure 8 complements the operator scatter in the main paper. The median operator-level speedup



(a) Autotune-mode speedup distribution across operators.



(b) Per-category roofline utilization R (Autotune-mode).

Figure 8: RQ1 extension: speedup distribution and per-category roofline utilization. **[Keren]: CuTile->cuTile**

is $1.95\times$ for Triton and $1.58\times$ for cuTile. 9 Triton implementations and 12 cuTile implementations remain below PyTorch. The largest operator-level speedups for both DSLs approximately $36\times$ on `bitonic_sort`, whose PyTorch reference is an eager multi-pass bitonic network rather than `torch.sort`. The large speedup therefore reflects, in part, the overhead of the PyTorch reference execution path rather than only the efficiency of the DSL backends. This example illustrates why the speedup distribution should be interpreted together with the corresponding PyTorch baseline implementation.

Table 3 shows that the software and hardware utilization views are complementary. Point-wise operators obtain the highest category-level roofline utilization and exhibit many near-parity Triton and cuTile results. Reduction / Normalization kernels also achieve strong speedups over PyTorch, often because the DSL kernels fuse operation or specialize a generation reduction template. Data Layout operators show the largest speedups over PyTorch but much lower roofline utilization, because the category contains multi-pass sorting and selection workloads whose PyTorch references and DSL algorithms have different launch and traffic structures. Stencil / Convolution and Matrix Multiplication / Attention remain the least utilized categories. Their arithmetic work is substantial, but performance is frequently limited by vendor-library gaps, dynamic indexing, state placement, or pipeline formation rather than peak Tensor Core throughput alone.

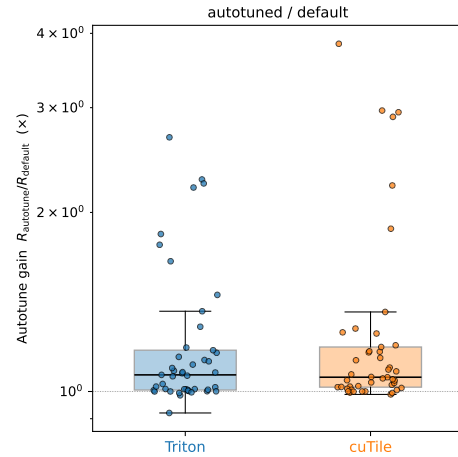


Figure 9: Triton and cuTile autotune gain $R_{\text{autotune}}/R_{\text{default}}$ per operator.

Triton has the high category-level speedup in all five groups, but this does not imply that cuTile is uniformly weaker inside a category. cuTile wins the most informative regular tiled cases, including `matmul_fp32_fp16_fp8`, `flash_attention`, and `jacobi_stencil_2d`. Its category aggregates are instead reduced by a smaller number of large deficits on irregular indexing, small reductions, sparse traversal, and runtime-controlled loops. This asymmetry is examined in Appendix D.3.

D.2 RQ2: Per-operator Autotuning Gains and Roofline Headroom

The suite-level geometric-mean tuning gain is $1.18\times$ for Triton and $1.22\times$ for cuTile, while the medians are $1.07\times$ and $1.06\times$. The larger cu-

Category	Speedup over PyTorch Triton	Speedup over PyTorch cuTile	Roofline R Triton	R cuTile	T wins	C wins
Point-wise	2.07×	1.84×	0.733	0.657		11
Reduction/Norm	2.56×	2.05×	0.337	0.269		9
Data Layout	3.37×	2.79×	0.139	0.115		6
Stencil/Conv	1.22×	0.86×	0.093	0.066		5
Matrix Mult./Attn	1.22×	0.79×	0.118	0.076		6

Table 3: Category-level geometric means under autotuned mode. The last two columns count direct Triton and cuTile wins inside each category.

Metric	Triton	cuTile
Default suite geometric-mean R	0.209	0.160
Autotuned suite geometric-mean R	0.248	0.194
Geometric-mean tuning gain	1.18×	1.22×
Median tuning gain	1.07×	1.06×
Autotuned $R \geq 0.8$	7/45	5/45
Autotuned $R \geq 0.9$	2/45	1/45

Table 4: Aggregate RQ2 results.

Tile geometric mean is driven by several high-gain operators rather than a uniformly larger shift. The strongest Triton gains are `argmax` (2.67×), `histogramming` (2.27×), `3d_conv` (2.24×), and `2d_conv` (2.20×). The strongest cuTile gains are `linear_self_attention` (3.84×), `1d_conv` (2.97×), `2d_conv` (2.95×), `argmax` (2.90×), and `3d_conv` (2.22×).

The autotuned result is the best measured candidate in the declared search space and is not clamped to the manually selected default. 4 Triton operator-level results and four cuTile results are therefore slightly below their defaults. 7 of these eight regressions are within approximately 1.5% of parity. The main exception is Triton `top_k_selection`, whose tuned aggregate is 0.92× its default. We retain these values rather than replacing them with the default because they accurately describe the implemented tuning protocol.

The results separate two kinds of headroom. Configuration headroom is recovered by changing tile shape, warp count, pipeline stages, or occupancy. Backend headroom remains when the source implementation and compiler continue to generate an expensive instruction path, an unsuitable reduction organization, excessive register or shared-memory state, or an ineffective software pipeline. The small number of operators above 80% utilization after tuning shows that the second kind of headroom dominates much of the suite.

D.3 RQ3: Cross-operator Profiling Summary

We organize the profiling results by the primary mechanism that best explains the performance difference between Triton and cuTile. Shared memory bank conflict is analyzed separately in Appendix D.3.7, since it can appear as a secondary hardware effect across multiple mechanisms.

D.3.1 Per-operator Diagnosis Map

Table 5 summarizes the primary diagnosis for each operator. The ratio $Q_o = S_{o,\text{Triton}}/S_{o,\text{cuTile}}$ is computed over the complete autotuned sweep, with values above one favoring Triton. The PyTorch path is included to distinguish practical software speedup from the direct backend diagnosis.

D.3.2 Dynamic Indexing and Gather / Scatter Lowering

Recurring mechanism. The strongest Triton advantages in this class occur when a logical tile is addressed through runtime-computed or non-affine indices. Triton represents the access as a pointer tensor and attaches its predicate directly to `tl.load` or `tl.store`. In the profiled cuTile implementations, the same access is often materialized as index and predicate tiles and lowered through `ct.gather` and `ct.scatter`. The resulting path may include integer division and modulo, bounds handling, tile bookkeeping, shared-memory staging, and element-oriented memory operations. The current profiles allow this mechanism to be separated from memory volume: in the strongest cases, the kernels issue the same useful loads while their instruction streams differ by an order of magnitude.

Weight dequantization. The two DSL implementations of `weight_dequant` contain the same flattened index expressions, `row = offsets/N`, `col = offsets mod N`, and the corresponding scale-tile coordinates. Triton loads the scale through a directly computed pointer, whereas cuTile uses `ct.gather`. In the BF16 maximum case,

Operator	Primary diagnosis	PyTorch path	Q_o
<i>Dynamic indexing and gather/scatter lowering</i>			
weight_dequant	Index/gather	Eager	1.73×
destindex	Index/scatter	ATen	2.10×
1d_conv	Implicit GEMM indexing	Vendor	1.76×
2d_conv	Implicit GEMM indexing	Vendor	1.56×
3d_conv	Implicit GEMM indexing	Vendor	2.19×
2d_max_pooling	Gather lowering	ATen	1.30×
gaussian_blur	Gather lowering	Vendor	1.21×
radix_sort	Scatter lowering	Vendor	1.87×
<i>Reduction granularity and primitive availability</i>			
cross_entropy	Reduction granularity	Eager	1.41×
moe_topk_gating	Reduction granularity	ATen	1.96×
top_k_selection	Primitive availability	ATen	1.13×
histogramming	Privatization/atomic primitive	ATen	1.62×
softmax	Reduction/normalization lowering	ATen	1.52×
<i>Register/SMEM placement and pipeline scheduling</i>			
matmul_fp32_fp16_fp8	Tensor pipeline scheduling	Vendor	0.79×
flash_attention	State placement	Vendor	0.94×
block_sparse_attention	SMEM staging/reuse mismatch	Compiled	2.69×
batch_normalization	Register/SMEM placement	ATen	1.11×
jacobi_stencil_2d	Reusable SMEM tile	Eager	0.87×
linear_self_attention	Runtime loop and stage pipeline	Mixed	2.02×
streamk_matmul	First wave runtime loop	Vendor	2.05×
batched_matmul	Register/SMEM pressure	Vendor	1.13×
mean_reduction	Dtype specific register pressure	ATen	1.14×
flash_decode	Runtime loop dependency	Eager	3.38×
<i>Dtype dependent exposure of fixed overhead</i>			
sigmoid	Fixed issue cost	ATen	1.31×
swiglu	Fixed issue cost	Eager	1.23×
relu	Fixed issue cost	ATen	1.06×
mul2	Fixed issue cost	ATen	1.05×
dequantize_rowwise	Load width/conversion cost	Eager	1.15×
layernorm	Fixed issue cost + wider write back	ATen	1.21×
l2_norm	Fixed loop/conversion cost	Eager	1.15×
rmsnorm	Fixed template cost	ATen	1.07×
<i>Backend parity and control cases</i>			
matrix_copy	Parity control	ATen	0.99×
matrix_transpose	Parity control	ATen	1.00×
quantize_global	Parity control	ATen	1.01×
interleave	Parity control	Eager	1.02×
reverse_array	Parity control	ATen	1.01×
matmul_int8	Hardware path control	Emulation	1.02×
vector_add	Parity control	ATen	1.01×
leaky_relu	Parity control	ATen	1.00×
dropout	Fusion control	Eager	1.01×
fused_activation	Fusion control	Eager	1.01×
bitonic_sort	Matched network control	Eager	1.00×
argmax	Reduction parity	ATen	0.97×
kl_divergence	Streaming parity	Eager	0.96×
rope	Memory floor parity	Eager	1.14×

Table 5: Primary performance diagnosis for all TileBench operators.

both kernels execute 3.686 million global-load instructions with the same 2.667 sectors per request. The useful memory work is therefore matched. Nevertheless, cuTile executes 133.1 million warp instructions compared with 21.5 million for Triton, a 6.19× expansion. Integer address and predicate instructions alone account for approximately 74.0 million cuTile executions versus 3.89 million for Triton, or 19.0×.

The SASS explains this expansion. cuTile retains repeated SHF.R.U32.HI sign-bit extraction and correction sequences around the division and modulo operations even though the offsets and divisors are nonnegative. Triton hoists the same coordinate calculation into a much shorter affine sequence. The cuTile kernel consequently reaches 87.7% ALU-pipeline activity, uses 124 registers per thread and 16.4 KB of shared memory per CTA, and achieves 23.5% occupancy. Triton uses 32 reg-

isters, no shared memory, and reaches 85.0% occupancy while remaining long-scoreboard dominated. The resulting CSV latency ratio is 2.755× in BF16. This is a compiler-lowering gap around the index expression, not a difference in memory traffic or operator algorithm.

Dynamic scatter in DestIndex. destindex provides the complementary scatter case. The two DSLs use the same flat elementwise decomposition and launch separate kernels for the non-RoPE and RoPE regions. Triton preserves the contiguous feature coordinate inside each dynamically selected destination row. Its main INT8 kernel therefore emits a 128-bit global store that covers 16 adjacent elements and records 16 sectors per request. cuTile lowers ct.scatter element by element, records one sector per request, and executes 16 times as many global-store instructions. The corresponding load path also exposes the difference in vectoriza-

tion, with approximately 8.5 sectors per request for Triton and 1.78 for cuTile in the dominant kernel.

Summed over both launches, cuTile executes 90.3 million warp instructions compared with 5.24 million for Triton, a $17.2\times$ expansion. Its occupancy is actually higher, approximately 68% versus 36%, while ALU-pipeline activity rises to 75.5% in the main kernel. Thus, the $2.942\times$ INT8 latency gap is not an occupancy shortage. It comes from the instruction issue and transaction structure of the general scatter path. The same fixed scatter management cost explains why the gap increases from $1.52\times$ in FP32 to approximately $2.52\times$ in FP16/BF16 and $2.94\times$ in INT8.

Dimensional scaling in convolution. The 1d_conv, 2d_conv, and 3d_conv implementations form a dimensional case study of the same mechanism. PyTorch dispatches to cuDNN, while both DSLs used a matched implicit-GEMM formulation. Every reduction tile must reconstruct spatial and kernel coordinates, check boundaries, generate indirect input and weight addresses, load the operands, and only then issue MMA. Triton constructs pointer tiles and applies the mask to `tl.load`; cuTile materializes linear coordinates and reaches the operands through `ct.gather`.

The dynamic-instruction expansion grows with the dimensionality of the coordinate reconstruction. In FP16, the cuTile-to-Triton ratios are $2.41\times$, $2.32\times$, and $3.87\times$ for 1D, 2D, and 3D. In FP32, they are $1.85\times$, $1.67\times$, and $2.62\times$. Their latency follows the same direction, take maximum input case as examples, cuTile requires $1.79\times$ and $1.73\times$ the Triton latency in 1D FP16 and FP32, $1.45\times$ and $1.50\times$ in 2D, and $2.58\times$ and $1.82\times$ in 3D. Across the family, ALU activity reaches approximately 60% – 78%, while tensor pipeline activity remains only about 0.9% – 15%. The tensor cores are waiting on the `im2col` address path rather than limiting throughput.

The MMA instruction family varies with the winning tile and therefore does not provide a family-wide explanation. Triton FP32 instead stages operands through LDGSTS and exhibits substantial shared memory bank conflicts. Using the normalized conflict metric C_{LSU} defined in Appendix D.3.7, the scores are 54.5, 51.9, and 79.6 for 1D, 2D, and 3D convolution, respectively, compared with 5.3, 6.8, and 3.0 in FP16. These conflicts reduce Triton’s FP32 margin but do not overturn the dominant result: the cuTile gather path

executes substantially more address, predicate, and staging work at every dimensionality.

cuDNN avoids much of this generic reconstruction through convolution-specific iterators and layouts. At the largest FP16 inputs, PyTorch is approximately $1.9\times$, $5.5\times$, and $9.8\times$ faster than Triton for 1D, 2D, and 3D convolution. This is a vendor-algorithm advantage, whereas the direct Triton and cuTile difference isolates how the two DSLs lower the same implicit-GEMM structure.

Radix scatter and supporting cases. Triton and cuTile use the same 2-bit, 16-pass LSD algorithm for `radix_sort`. The PyTorch advantage comes from CUB’s different radix-sort algorithm, but the direct DSL gap is concentrated in the matched scatter stage. Scatter accounts for 88.9% of the NCU duration difference. Compared with Triton, cuTile executes $2.31\times$ as many scatter-stage instructions, increases the register count from 64 to 108 and shared memory from 8.2 to 33.8 KB, and lowers achieved occupancy from 44.9% to 23.0%, while the two kernels retain the same load and store sectors per request. The extra cost is therefore lowering and resource state rather than a different memory pattern.

`2d_max_pooling` and FP16 `gaussian_blur` provide smaller supporting examples. Their cuTile-to-Triton instruction ratios, $1.35\times$ and $1.47\times$, closely match their latency ratios, $1.39\times$ and $1.49\times$.

Takeaway. Runtime indexing is not expensive merely because it is irregular. The decisive question is whether the compiler preserves a contiguous subdimension while representing the dynamic coordinates. Triton often keeps that structure visible in a pointer tensor. The current cuTile gather and scatter paths may instead materialize a general element-wise operation, producing longer integer, predicate, staging, and memory-instruction sequences. This difference can turn a nominally streaming or tensor core kernel into an address-generation bottleneck.

D.3.3 Reduction Granularity and Primitive Availability

Recurring mechanism. Small row-wise reductions expose a mismatch between useful work and execution granularity. Triton can map a short reduction to one warp and keep intermediate state in registers and shuffles. CuTile often use a 128-thread CTA and a shared-memory tree for cross-warp exchange, which raises shared-memory, barrier, and

control-flow cost even when the useful reduction is small. A second source is primitive availability: when only one DSL exposes a top-k or histogram primitive, the result measures programming model expressibility as well as compiler lowering.

Cross entropy and MoE gating. For cross_entropy, Triton’s winning kernel uses 1 warp per row and generates no shared-memory load, store, or barrier instructions. cuTile uses 4 warps and lowers the max and sum operations through a shared-memory tree. In FP16, the cuTile kernel executes approximately 780 instructions per row, compared with 226 for Triton, and adds about 10 shared loads, 10 shared stores, and 20 barrier executions per row. The two kernels execute the same number of MUFU.EX2 instructions, so the arithmetic path is matched. Although cuTile executes approximately $3.45\times$ as many instructions per row as Triton, the resulting latency gap is only $1.54\times$, because cuTile sustains much higher occupancy, approximately 82% versus 27% for Triton.

moe_topk_gating amplifies the same granularity effect. Triton again selects a single-warp kernel and uses no shared-memory reduction traffic. cuTile executes approximately $6.5\times$ as many instructions per row, performs 22 shared loads and 30 shared stores per row, and issues 4 times as many global-load instructions. Across FP16, BF16, and FP32, this produces a maximum case gap of $2.17\times$ to $2.19\times$.

Histogram privatization. The current Triton and cuTile histogramming winners use the same global-memory privatization strategy. Both execute exactly 2,097,152 global atomic additions, and neither uses shared-memory atomics. The direct gap therefore comes from the operand contract and live tile state around the atomic operation. Triton folds the address and predicate into a scalar `tl.atomic_add`. cuTile materializes row-index, bin-index, validity, and update tiles for `ct.atomic_add`. Relative to Triton, cuTile executes 72.97 versus 50.69 million instructions, uses 128 versus 40 registers per thread, and achieves 10.8% versus 21.6% active warps. For the largest case, cuTile’s latency is $1.78\times$ that of Triton. PyTorch follows a third strategy, shared memory privatization inside a vendor path, so its crossover across input sizes reflects a library algorithm effect rather than a direct compiler comparison.

Top-k primitive availability. top_k_selection is not a strictly matched algorithmic comparison because Triton provides `tl.topk`, whereas cuTile has no native top k primitive and instead uses a fully unrolled bitonic network through shared memory. At $K = 16$, Triton is $1.49\times$ faster than cuTile, while at $K = 1024$, cuTile is $1.58\times$ faster than Triton and its largest stage executes fewer instructions than the corresponding Triton stage. Across the full sweep, Triton is $1.13\times$ faster than cuTile in geometric mean. The crossover therefore reflects primitive availability and the implementation strategies enabled by each programming model, rather than compiler code generation under an identical algorithm.

Softmax. For softmax, the two backends use the same fast exponential path. In FP16, both execute exactly 1,351,680 MUFU.EX2 instructions. cuTile’s extra work instead comes from reciprocal and normalization lowering: its total MUFU instruction counts are 2.007 million and 1.393 million, the FFMA counts are 12.9 million and 8.4 million, and the barrier counts are 2.96 million and 0.246 million for cuTile and Triton, respectively. cuTile also executes substantially more branches. Consequently, cuTile’s latency is $1.82\times$ that of Triton in FP16 and $1.24\times$ in FP32. We therefore attribute the performance gap to reduction, normalization, and control-flow lowering, rather than a different exp instruction.

Takeaway. Reduction performance depends on whether the programming model can express the natural granularity of the problem. For short fixed reductions, a CTA-wide shared-memory tree adds work that tuning cannot remove. For top-k and histogram, the more fundamental question is which primitive or privatization strategy the language exposes. These cases therefore separate matched reduction lowering from programming model expressibility.

D.3.4 Register / SMEM Placement and Pipeline Scheduling

Recurring mechanism. When addressing is regular, the main backend difference shifts from index generation to on-chip state placement and pipeline formation. Triton often keeps small intermediate values in registers and exposes explicit warp and stage parameters. cuTile more often uses shared-memory tiles managed by the compiler. Storing in-

intermediate data in shared memory introduces staging and synchronization, but it can support a larger reusable tile and reduce loop count. The same strategy can therefore win on dense reuse and lose when the intermediate data is sparse or only lightly reused.

Regular GEMM control case. `matmul_fp32_fp16_fp8` removes the irregular addressing present in convolution. Both DSLs use TMA and `tcgen05`. Across FP32, FP16, and FP8, tensor pipeline activity ranges from 79% to 81% for cuTile and from 52% to 69% for Triton. At $M = N = 4096$ and $K = 20480$, cuTile is $1.55\times$, $1.12\times$, and $1.22\times$ faster than Triton in FP32, FP16, and FP8, respectively.

The FP32 result is a shared memory capacity case. Triton’s winner uses a $128 \times 128 \times 32$ tile with 98.4 KB of shared memory, while cuTile sustains $256 \times 256 \times 64$ with approximately 229.7 KB. The smaller Triton tile causes 21.47 GB of TMA loads, compared with 10.74 GB for cuTile. cuTile consequently keeps the tensor pipeline active for 81.2% of cycles rather than 52.4% (Triton). In FP16 and FP8, both backends use the same large tile and the TMA bytes match, leaving a smaller scheduling gap. This control demonstrates that cuTile’s tensor core backend is not inherently weak; on a regular GEMM, its state layout can feed the compute pipeline more effectively.

Dense and sparse attention. `flash_attention` shows the beneficial side of a large shared memory tile. Triton selects 128×64 , uses 128 registers per thread and 96.6 KB of shared memory, and executes approximately 10.77 billion warp instructions. cuTile selects 128×128 , uses 168 registers and 224.4 KB, and executes approximately 4.45 billion warp instructions. The larger N tile halves the K/V loop count and makes cuTile $1.10\times$ faster than Triton at the maximum case, despite cuTile’s lower occupancy and 3.9 times more barrier and synchronization executions than Triton. Both DSLs use TMA and `tcgen05`. PyTorch remains faster through a cuDNN warp-specialized producer-consumer schedule that reaches approximately 79% tensor pipeline activity, compared with 32% and 35% for the Triton and cuTile, respectively.

The same cuTile strategy of using a large share memory tile, approximately from 221 to 230 KB per CTA, is less effective on `block_sparse_attention`, where each staged

sparse block provide less data reuse. Compared with Triton, cuTile executes $2.14\times$ as many instructions, performs about 14.0 million barrier and synchronization operations rather than 0.47 million, and uses 221 KB of shared memory per CTA. Its maximum case CSV latency is therefore $3.08\times$ higher. Notably, cuTile uses `tcgen05` while Triton uses legacy HMMA in this winner, yet Triton remains substantially faster. The dominant issue is therefore the additional shared memory staging, synchronization, and sparse block traversal, rather than the choice of MMA instruction family.

Multi-stage attention. The `linear_self_attention` implementation contains two activation launches, a KV GEMM, a column reduction kernel, and an output GEMM. All GEMM stages use TMA. cuTile lowers both GEMMs to `tcgen05`; Triton’s 64-row output GEMM also uses `tcgen05`, while its 32-row KV winner remains on legacy HMMA. At the maximum case, cuTile requires $2.19\times$ the Triton latency, so access to `tcgen05` alone does not explain the result.

The remaining cost comes mainly from the column reduction and the scheduling of the KV loop. The column reduction kernel launches only 8 CTAs in both backends and accounts for about 57% of the summed NCU kernel duration in Triton and 55% in cuTile. For this kernel, cuTile takes 274.0 μs , while Triton takes 115.4 μs . cuTile also executes $1.75\times$ as many instructions and exhibits a deeper long-scoreboard dependency. In the KV GEMM, cuTile uses a runtime while loop without an effective software pipeline, launches only 16 CTAs and records approximately 46 cycles of long-scoreboard stall per issued instruction. Triton instead use a staged loop with `num_stages=3`. The remaining performance gap therefore comes mainly from limited parallelism in the column reduction and ineffective pipeline of the runtime K loop.

Low-parallelism runtime loops. `flash_decode` shows the largest maximum case gap, with cuTile requires $3.72\times$ the Triton latency. Both kernels launch only 16 CTAs and use no shared memory, TMA, or tensor core instructions. cuTile executes approximately 1.012 million warp instructions, compared with 0.487 million for Triton, or $2.08\times$ as many. cuTile also records 16.69 cycles of long scoreboard stall per issued instruction, compared with 9.22 cycles for

Triton. The performance gap therefore combines a $2.08\times$ expansion in the cuTile instruction stream with a deeper dependency chain in a grid with limited parallelism to hide latency. This case reflects software pipeline and dependency handling rather than register or shared memory placement.

`streamk_matmul` is strongly shape dependent. Its implementation contains two main kernels: The `first_wave` kernel distributes a subset of output tiles across the available SMs and accumulates partial results through atomic updates, while `full_tiles` processes the remaining complete output tiles. Both Triton and cuTile use this same decomposition.

At the largest input, cuTile requires only $1.04\times$ to $1.06\times$ the Triton latency across the evaluated dtypes, so the two backends are close to parity. The `full_tiles` kernels accounts for most of the work and is actually faster in cuTile, where it reaches higher tensor pipeline activity. The remaining difference is concentrated in `first_wave`.

In `first_wave`, cuTile uses nested runtime while loops. It executes about 12.0 million barrier and synchronization operations, compared with 1.16 million for Triton. cuTile also records approximately 16 cycles of wait stall per issued instruction, compared with about 1.4 cycles for Triton. Triton instead uses a staged inner loop with `num_stages=3`. At smaller M , the `first_wave` kernel represents a larger fraction of the total work. For example, at $M = 1024$, cuTile is approximately $2.05\times$ and $2.06\times$ slower than Triton in FP16 and BF16, respectively. As M increases, `full_tiles` dominates the execution, and its cuTile advantage largely offsets the slower `first_wave` kernel.

Other register and shared memory trade-offs. cuTile’s larger reusable tile benefits `jacobi_stencil_2d`. Triton uses a single-row tile and no shared memory in FP32, while cuTile loads a 4×512 region into shared memory and reuses the vertical halo. cuTile is approximately $1.24\times$ faster than Triton in FP32 and $1.23\times$ faster in FP16/BF16. `batch_normalization` shows a dtype-dependent reversal: Triton is $1.56\times$ to $1.60\times$ faster than cuTile in FP16/BF16, but cuTile is $1.20\times$ faster than Triton in FP32 because Triton keeps the block accumulation state in registers, which becomes more expensive in FP32, while cuTile continues to store the intermediate state in shared memory and avoid the same increase in

register pressure.

The opposite outcome appears in FP32 `batched_matmul`. Both backends use TMA and `tcgen05` with equal TMA load volume, but cuTile uses 255 registers per thread and 180.4 KB of shared memory, reaches only about 10.6% occupancy, and executes $2.49\times$ as many instructions as Triton. Its final result handling through scalar shared memory also yields $C_{LSU} = 47.88$. For the largest case, cuTile’s latency is $1.76\times$ that of Triton. In `mean_reduction`, the register pressure explanation applies only to the narrow winner: in FP16, register use is approximately 74 and 31 per thread, while occupancy is 32.7% and 89.8% for cuTile and Triton, respectively. The current FP32 cuTile winner uses only 30 registers per thread, and its latency is close to Triton’s, so we do not generalize the register claim across dtypes.

Takeaway. The choice between registers and shared memory depends on the amount of intermediate data and how often it is reused. Registers are effective when the intermediate state is small, while shared memory can support larger reusable tiles and reduce the number of loop iterations. However, shared memory introduces additional data movement and synchronization, which becomes costly when the loaded data is reused only a few times or when synchronization occurs repeatedly inside runtime loops. The resulting performance therefore depends on data reuse, register and shared memory usage, loop structure, and the available parallelism for hiding instruction and memory latency.

D.3.5 Dtype-dependent Exposure of Fixed Scheduling

Recurring mechanism. A common dtype trend can be summarized by

$$T \approx \max(T_{\text{memory}}, T_{\text{issue}}). \quad (7)$$

Moving from FP32 to FP16, BF16, or INT8 reduces the memory time per element, but it does not necessarily reduce tile index and boundary book-keeping, predicates, dtype conversions, transcendental instructions, or control flow. An instruction path hidden by HBM latency in FP32 can therefore become visible at a narrower dtype.

Element-wise functions. `sigmoid` is a clear example. In FP16, Triton and cuTile execute the same number of MUFU.EX2 instructions, but cuTile executes $1.77\times$ as many total instructions as Triton. In particular, cuTile executes 18.0 million

FFMA family instructions compared with 10.9 million for Triton, and 1.56 million IMAD instructions compared with 0.20 million for Triton. As a result, cuTile requires $1.47\times$ the Triton latency for FP16. In FP32, however, the two DSLs differ by less than 0.2% because both kernels reach the memory floor.

swiglu shows the same pattern: In FP16, cuTile executes $1.44\times$ as many instructions as Triton and requires $1.33\times$ the Triton latency. In FP32, cuTile requires only $1.02\times$ the Triton latency. For INT8 relu, both backends issue the same 128-bit global loads and stores, yet cuTile executes $1.85\times$ as many instructions as Triton and requires $1.34\times$ the Triton latency. Their FP32 latencies differ by less than 1%. mul2 is the lightest example of the same pattern: in INT8, cuTile executes $1.65\times$ as many instructions as Triton and requires $1.14\times$ the Triton latency, while the two DSLs remain near parity in FP32.

Conversion and normalization. dequantize_rowwise exposes transaction width in addition to fixed issue cost. Triton uses 128-bit global memory load instructions for the quantized input, whereas cuTile uses 64-bit loads for the same data and therefore executes twice as many load instructions. Overall, cuTile executes $2.13\times$ as many instruction as Triton and requires $1.21\times$ the Triton latency in INT8.

The normalization kernels provide supporting examples. For l2_norm, cuTile requires approximately $1.28\times$ the Triton latency in FP16, but only $1.03\times$ in FP32. rmsnorm shows the same trend with a smaller gap. For layernorm, cuTile requires $1.61\times$ the Triton latency in FP16, the largest narrow dtype gap among these normalization cases. This gap is not caused by instruction overhead alone: cuTile winner also writes approximately 49% more DRAM bytes than Triton in FP16 because of additional FP32 intermediate write traffic. We therefore attribute the FP16 gap to both additional instruction execution and extra FP32 intermediate write traffic in cuTile.

Takeaway. The dtype trend is mainly caused by a changing balance between memory cost and instruction cost. For FP32, memory traffic often dominates and hides differences in index calculation, conversion, and control flow. As the dtype becomes narrower, memory traffic decreases while much of this instruction work remains, making the backend difference visible in latency. This explains why several operators are

close to parity in FP32 but show a larger Triton advantage in FP16, BF16, or INT8.

D.3.6 Cases with Similar DSLs Performance

Recurring mechanism. Across the complete autotuned sweep, Triton and cuTile differ by less than 5% on 14 operators and by less than 10% on 17 operators. The 14 operators assigned to this class are selected by their profiling behavior rather than by a fixed numerical threshold. For these operators, neither backend shows a recurring mechanism that consistently dominates the direct comparison.

Regular memory movement. matrix_copy, matrix_transpose, quantize_global, interleave, reverse_array, vector_add, and leaky_relu provide regular memory movement or point-wise controls. For matrix_copy, the cuTile latency is between $0.99\times$ and $1.02\times$ the Triton latency across all dtypes. The two backends move the same amount of DRAM data and both issue 16 sectors per request. For matrix_transpose, both implementations use shared memory to perform the same matrix reordering, and their latencies differ by less than 2%. These cases show that regular memory access does not introduce a large systematic cost in either programming model.

rope provides a useful memory bound control. In the profiled cases, cuTile executes approximately $3\times$ as many instructions as Triton, yet both backends sustain approximately 5.9 to 6.4 TB/s and their latencies differ by less than 4%. The memory cost therefore hides the additional cuTile instructions. For this reason, rope is better interpreted as a parity case than as evidence that instruction overhead must always produce a latency gap.

Fusion controls. dropout and fused_activation show that a large speedup over PyTorch can coexist with nearly identical Triton and cuTile performance. For fused_activation, cuTile executes $2.41\times$ as many instructions as Triton, but both backends move approximately 350 MB in about $54\mu\text{s}$ and sustain roughly 6.4 TB/s. The additional cuTile instructions are therefore hidden by memory bandwidth. The speedup of both DSL implementations over PyTorch mainly comes from fusion and reduced intermediate traffic, rather than from an advantage of one DSL over the other.

Algorithm and hardware controls. The Triton and cuTile implementations of `bitonic_sort` execute the same approximately 300 pass sorting network and have nearly identical latency. Both implementations are dominated by the repeated memory movement across the serial passes rather than by a backend specific code generation effect. `argmax` is also close in the operator aggregate, although at the maximum FP32 case cuTile is $1.12\times$ faster than Triton. Similarly, cuTile is approximately $1.09\times$ faster than Triton for `kl_divergence`. We retain these cases because the differences remain small and do not form a consistent advantage for either backend across this group. `matmul_int8` provides the clearest hardware control. At the maximum case, Triton takes 0.3272 ms and cuTile takes 0.3289 ms, so the cuTile latency is only $1.005\times$ the Triton latency even though the generated MMA paths differ substantially. Triton uses legacy IMMA with approximately 20.97 million IMMA executions, 28.8 million permutation and conversion instructions, LDSM staging, and TMA loads. cuTile uses UTCIMMA through the `tcgen05` path, together with TMA loads, shared memory and TMEM staging, and about 11.2 million barrier and synchronization executions. Both kernels achieve approximately 10.8% occupancy and spend much of their execution feeding operands into the MMA path. PyTorch takes 3.4325 ms because its reference performs exact FP32 emulation rather than using the fused integer Tensor Core path. This case shows that a newer MMA instruction family does not necessarily reduce end to end latency when both kernels are limited by operand delivery and staging.

Takeaway. These controls show that neither cuTile nor Triton carries a large fixed performance penalty across all workloads. When both backends are limited by the same memory traffic, serial execution, reduction structure, or operand delivery cost, their latencies can remain close even when their generated instruction streams differ. The larger gaps discussed in the preceding sections therefore arise only when a workload exposes a specific difference in index generation, reduction strategy, register and shared memory use, pipeline scheduling, or dtype dependent instruction cost.

D.3.7 Shared-memory Bank-conflict Analysis

Bank conflicts can appear in operators from several diagnosis classes, so we analyze them separately

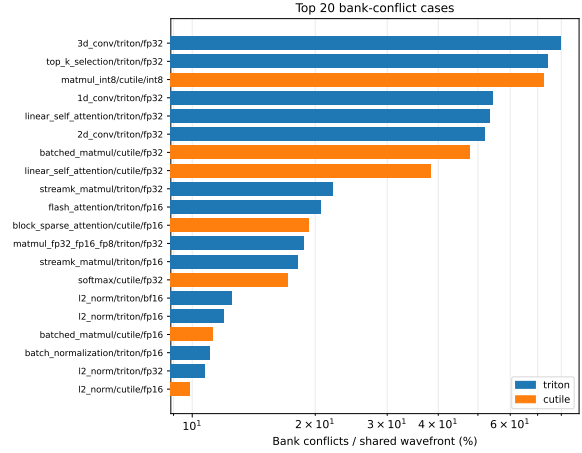


Figure 10: The 20 largest shared memory conflict scores among the 220 primary kernel profiles.

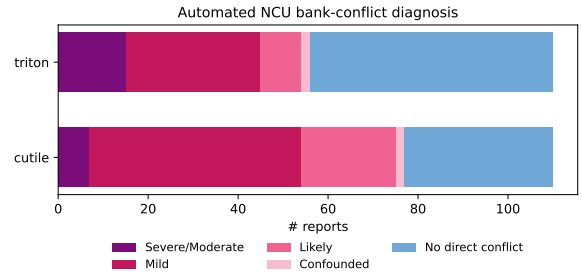


Figure 11: Shared memory conflict diagnosis counts for Triton and cuTile.

rather than treating them as another primary class. We define the normalized LSU shared memory conflict metric as

$$C_{\text{LSU}} = 100 \times \frac{B_{\text{LSU}}}{W_{\text{LSU}}}, \quad (8)$$

where B_{LSU} and W_{LSU} are the NCU LSU shared memory bank conflict and wavefront counters, respectively.¹ This metric is not a percentage of execution time and is not bounded by 100 in principle. We classify a profile as “Severe/Moderate” when $C_{\text{LSU}} \geq 10$, “Mild” when $1 \leq C_{\text{LSU}} < 10$, “Likely” when $0.1 \leq C_{\text{LSU}} < 1$, and “No direct conflict” when $C_{\text{LSU}} < 0.1$. We additionally use branch efficiency to identify cases where branch divergence may confound the bank conflict signal. If branch efficiency is available and falls below 98%, we label the profile “Confounded” before applying these thresholds.

All 220 primary kernel profiles provide a valid C_{LSU} value. Figure 11 contains 18 “Severe/Moder-

¹The corresponding NCU metrics are `l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum` and `l1tex__data_pipe_lsu_wavefronts_mem_shared.sum`.

ate" profiles, 74 "Mild" profiles, 37 "Likely" profiles, 4 "Confounded" profiles, and 87 profiles with "No direct conflict". Triton has 13 "Severe/Moderate" cases and 54 cases with "No direct conflict", while cuTile has 5 "Severe/Moderate" cases and 33 cases with "No direct conflict". cuTile has 70 "Mild" or "Likely" profiles, compared with 41 for Triton. Thus, cuTile has more "Mild" and "Likely", whereas Triton has more "Severe/Moderate" profiles as well as more profile with "No direct conflict".

The three largest C_{LSU} values in Figure 10 occur in 3d_conv/Triton/FP32 at 79.63, top_k_selection/Triton/FP32 at 73.97, and matmul_int8/cuTile/INT8 at 72.62. The top 20 contain 13 Triton and seven cuTile kernels. Inspection of the corresponding source and SASS identifies shared memory store layouts, shared memory used to store intermediate reduction results, and loading and arranging MMA operand in shared memory as the main sources of these conflicts. Representative examples include STSM/LDS operand staging in Triton 3D convolution, temporary shared memory loads and stores in Triton tl.topk, byte stores into the unpacked B tile in cuTile matmul_int8, temporary shared memory used for the column reduction in linear_self_attention, and scalar shared memory stores during final result handling in cuTile FP32 batched_matmul. Because large conflict values appear in both backends and across several diagnosis classes, bank conflict does not explain the overall Triton and cuTile performance ranking.

The clearest dtype pattern in Figure 12 appears in Triton convolution. For FP16, the C_{LSU} values are 5.30, 6.77, and 3.03 for 1D, 2D, and 3D convolution, respectively. For FP32, they increase to 54.51, 51.93, and 79.63. Other operators show weaker changes or the opposite direction, so this behavior is specific to these convolution kernels rather than a general FP32 effect.

The matched comparison in Figure 13 shows no consistent advantage for either backend. cuTile has a higher C_{LSU} value in 46 operator and dtype pairs, Triton has a higher value in 32 pairs, and both values are exactly zero in the remaining 32 pairs. Only linear_self_attention/FP32 has $C_{LSU} \geq 10$ in both backends. The cuTile profile is labeled Confounded because its branch efficiency is 95.17%.

Figure 14 provides two additional checks on the

interpretation of C_{LSU} . Branch efficiency is available for 133 of the 220 profiles, and only four are classified as "Confounded". Across all 220 profiles, the Spearman correlation between C_{LSU} and the issued versus executed IPC gap is only 0.205. These results indicate that neither branch divergence nor the IPC gap provides a general explanation for the observed conflict scores. More importantly, a high C_{LSU} value should not be interpreted directly as a latency penalty. We therefore use bank conflict only as supporting evidence when it agrees with the shared memory access pattern and the measured performance behavior.

D.4 RQ4: Iterative Refinement Trajectories

The LLM track allows at most ten refinement iterations. A backend freezes once it is verify-clean and reaches the stopping threshold, after which it incurs no additional token cost. Failed executed iterations remain part of the token total because compilation and correctness failures are part of the search process.

Triton produces at least one verify-clean implementation for all 45 operators with both models. cuTile fails to produce one for two GPT-5.5 operators and four Claude Opus 4.7 operators. The geometric-mean BestSpeedup@10, computed over positive verify-clean results, is $4.12\times$ for GPT-5.5 + Triton, $3.46\times$ for GPT-5.5 + cuTile, $3.57\times$ for Claude Opus 4.7 + Triton, and $3.16\times$ for Claude Opus 4.7 + cuTile. The token-efficiency result therefore reflects both lower token cost and stronger verify-clean performance on Triton.

Figure 15 illustrates why a single final score does not fully describe the refinement process. GPT-5.5 + Triton reaches a valid implementation early and peaks at approximately $2.64\times$ in iteration 8. Claude Opus 4.7 + Triton reaches approximately $1.73\times$ by iteration 9. GPT-5.5 + cuTile fails verification at iteration 0, later drops to approximately $0.76\times$, and reaches approximately $2.45\times$ by iteration 9. Claude Opus 4.7 + cuTile remains near $0.43\times$. The trajectories show that later refinement can improve both backends, but the final experiment does not vary the iteration budget and therefore does not establish whether a larger budget would eliminate the backend gap.

The LLM results quantify an empirical usability difference under the TileBench protocol. They do not isolate whether the cause is API expressibility, primitive availability, compiler constraints, documentation, public examples, or prior model

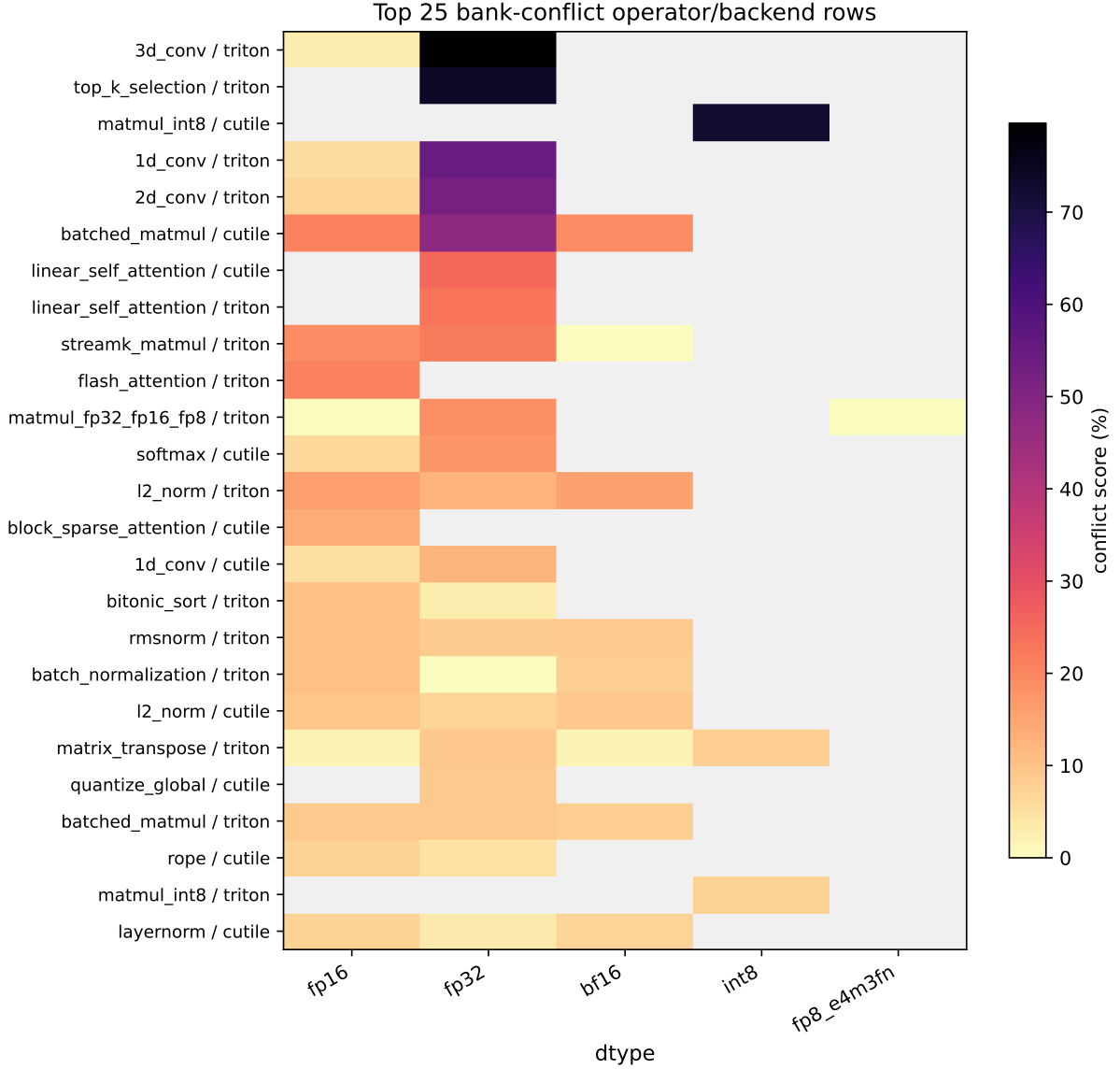


Figure 12: Shared memory conflict scores across dtypes for the operator and backend pairs with the largest values.

Model and backend	Verify-clean coverage	Faster than PyTorch	TokenCost@10	TokenEfficiency@10
GPT-5.5 + Triton	45/45	38/45	0.260M	15.83
GPT-5.5 + cuTile	43/45	36/43	0.331M	10.79
Claude Opus 4.7 + Triton	45/45	38/45	0.277M	12.90
Claude Opus 4.7 + cuTile	41/45	33/41	0.441M	7.61

Table 6: Final LLM-track coverage and geometric-mean token metrics. The faster-than-PyTorch denominator in the cuTile rows is the number of operators with at least one verify-clean implementation.

exposure. We therefore avoid treating model familiarity as a demonstrated causal explanation.

E Computational Budget

All experiments were run on a single NVIDIA B200 GPU with 180GB HBM3e memory. The main benchmark, autotuning, NCU report generation experiments consumed approximately 100

GPU-hours in total.

F LLM API Usage

For the iterative LLM code-generation track, GPT-5.5 consumed 18.68M API tokens with a total cost of \$279.05, while Claude Opus 4.7 consumed 21.09M API tokens with a total cost of \$191.66. In total, the LLM track consumed 39.77M API to-

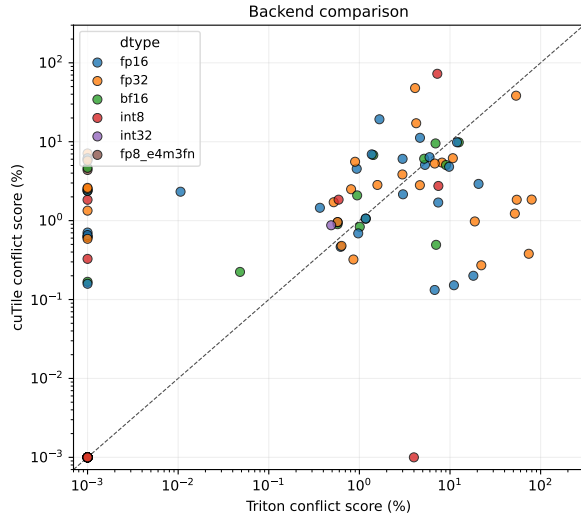


Figure 13: Matched Triton and cuTile shared memory conflict scores for all 110 operator and dtype pairs. Points above the diagonal have a higher cuTile score, while points below the diagonal have a higher Triton score.

kens and cost \$470.71. Because both evaluated LLMs are closed-source API models, their parameter counts are not publicly available.

G Artifact Sources and Licenses

TileBench uses TritonBench and LeetGPU as sources of operator coverage and task semantics. TritonBench is distributed under the Apache License 2.0. For TritonBench-derived components, we preserve the original attribution and license notices and document our modifications when applicable.

LeetGPU is distributed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International license (CC BY-NC-ND 4.0). This license permits non-commercial sharing with attribution, but does not permit redistribution of adapted material. Accordingly, TileBench does not redistribute LeetGPU source code, test files, or modified problem statements. For operators selected from LeetGPU challenge topics, we provide independently written PyTorch references, Triton implementations, cuTile implementations, and TileBench configuration files, and we cite the original LeetGPU artifact.

TileBench artifacts are newly written by the authors. Third-party artifacts, tools, and dependencies remain governed by their original licenses and terms of use.

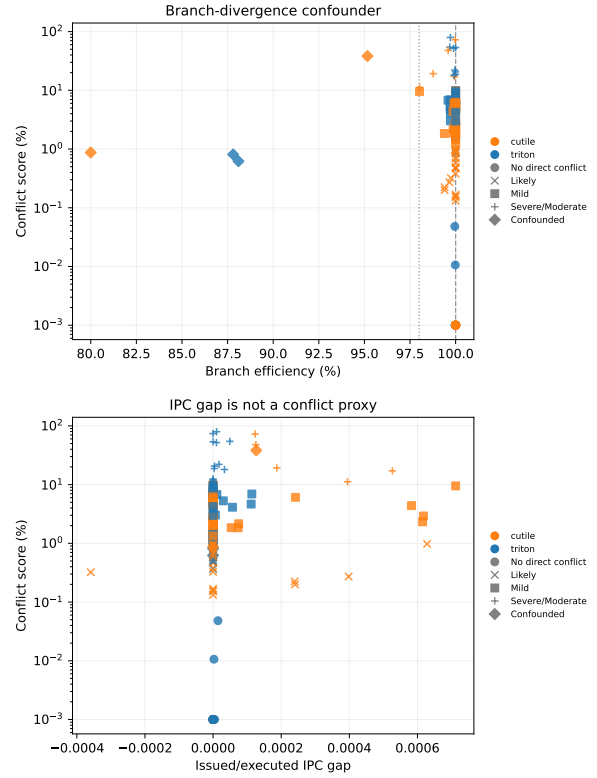


Figure 14: Additional checks for the bank conflict analysis. Left: conflict score versus branch uniformity for profiles that expose the branch metric. Right: conflict score versus the gap between issued and executed IPC.

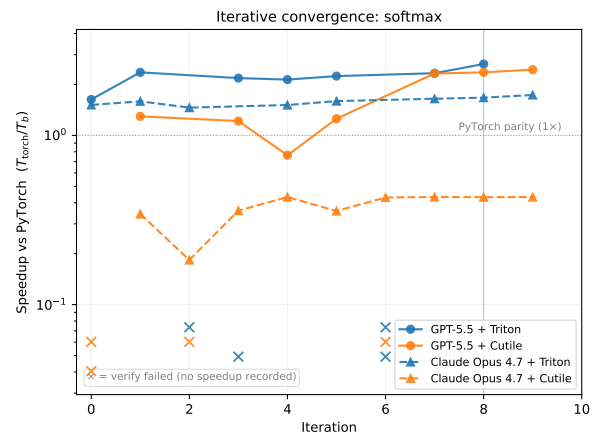


Figure 15: Iteration-level correctness-gated speedup trajectory for softmax. Frozen or failed backend iterations do not contribute a new valid speedup point.